

Transport of skips between recycling centers and treatment facilities

Sanne Wøhlk^{a,*}, Gilbert Laporte^{b,c}

^a CORAL - Cluster for Operations Research, Analytics, and Logistics, Department of Economics and Business Economics, Aarhus University, Fuglesangs Allé 4, DK-8210 Aarhus V, Denmark

^b HEC Montréal, 3000, Chemin de la Côte-Sainte-Catherine, Montréal, Canada H3T 2A7

^c School of Management, University of Bath, Claverton Down, Bath, BA2 7AY, United Kingdom

ARTICLE INFO

Keywords:

Transport
Waste
Pickup and delivery problems
Skip transport
Recycling centers
Variable neighborhood search

ABSTRACT

We study two versions of a skip pickup and delivery problem. In the first version, skips are picked up from recycling centers and, depending on their content, they are transported to specified treatment facilities where they are emptied before being returned to their origin. In the second version, the point of return of empty skips is flexible and determined by the skip type. We provide mathematical models for both versions of the problem and present a variable neighborhood search-inspired heuristic for solving them to near-optimality. Computational results are provided for real-life instances from four areas in Denmark.

1. Introduction

The problem studied in this paper originates from a corporate research agreement with a number of waste companies in Denmark, but is also relevant to many other contexts. It is the problem of picking up full skips, transporting them to a specific location where they are emptied, and returning them either to their origin or to another place where they are requested. Skips are metal containers typically used to hold debris from construction sites or material from recycling centers or treatment facilities. We present mathematical models for both versions of the problem along with a number of valid inequalities, and we develop a variable neighborhood search heuristic (VNS) to generate high-quality solutions on real-life instances.

In Denmark, residential waste is roughly collected through three very different systems. (1) Residual waste is collected directly from the curbside; depending on the county, this can cover residual waste only, or also a number of recyclables and organic waste. See Kiilerich and Wøhlk (2018) and Zbib and Wøhlk (2019) for more information on this type of collection. From an operations research point of view, curbside collection can be modeled as an arc routing problem (Corberán and Laporte, 2015; Mourão and Pinto, 2017; Corberán et al., 2021; De Maio et al., 2021). (2) In some counties, paper and glass are also collected through a bring system from so-called cubes, which are essentially mini-skips located in easily accessible locations (Bogh et al., 2014; Elbek and Wøhlk, 2016; Elbek, 2018). Collection from recycling cubes can be modeled as an inventory-routing problem (Andersson et al., 2010; Coelho et al., 2014; Archetti and Bertazzi, 2021). (3) Finally, most counties have a number of recycling centers, which is the topic

of this paper. A recycling center contains dedicated skips for 20 to 30 types of recyclable materials (textile, iron, old tires, etc.). Citizens go to the recycling centers by car and drop their stuff in the skips. To ensure correct sorting, these recycling centers are usually staffed to provide guidance to the citizens. There are 364 such recycling centers in Denmark, distributed among 98 counties, and they jointly receive around 8.6 thousands tons of material annually (Ekeröth, 2019). Fig. 1 shows a photograph of a recycling center.

A county usually has between two and 10 such recycling centers. When the skips are full, they are transported to a treatment facility which depends on the content of the skip. There the skip is emptied, after which it is typically returned to its origin. This transport generally takes place in the morning before the recycling center opens and the staff has generally decided the day before which skips to empty. This transport problem is the focus of this paper. Fig. 2 shows an example of the locations of recycling centers and treatment facilities for one of our data areas.

The operational practices differ between the counties. While some counties operate the transport themselves, others outsource it to a logistics provider. In some counties, the transport is planned centrally, while in others, each driver (or set of drivers) is assigned to a few recycling centers and handles all transport from these recycling centers. In some of the counties with central planning, skips need not necessarily be returned to their origin, but can be returned to another recycling center where a skip of the same type originates (in terms of size as well as some other characteristics). In this paper, we consider two versions of a central planning system: one with fixed return where each skip must

* Corresponding author.

E-mail addresses: sanw@econ.au.dk (S. Wøhlk), gilbert.laporte@cirrelt.net (G. Laporte).



Fig. 1. Photograph of a recycling center, provided by one of our industrial partners Reno Djurs I/S.

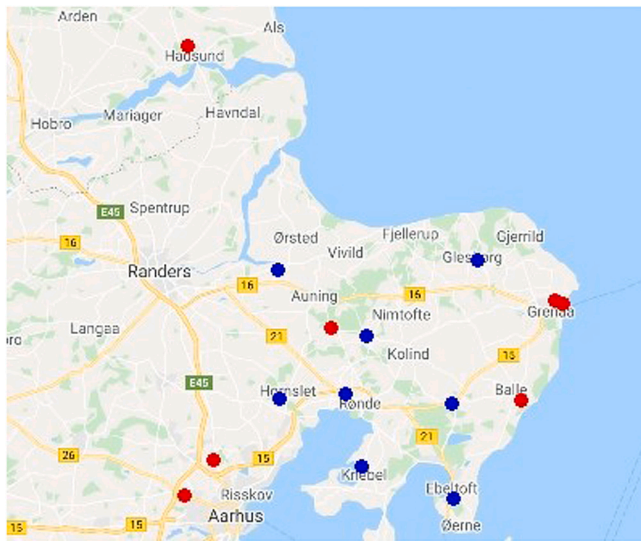


Fig. 2. The location of recycling centers (blue) and treatment facilities (red) in the C data sets from North and South Djurs municipalities in Denmark.

be returned to its origin, and one with flexible return. In both versions, transport is performed with a vehicle called tractor that can carry two skips simultaneously, and the skips can be loaded on and off the tractor in any order.

The relevance of our study is not limited to recycling centers in Denmark. Indeed it applies to any situation where skips are transported to dedicated sites to be emptied, as opposed to being emptied on-site. Several such examples are reported in the context of African cities (Yusuf et al., 2019; Tonny and Emmanuel, 2020; Richard et al., 2021). Besides collection of recyclable material, such situations occur, for instance, in the building industry (Rabbani et al., 2016).

The scientific contributions of the paper are the following: (1) We model and solve a complex problem arising in the transport and emptying of skips. (2) We present the **first mathematical model and valid inequalities for the problem**. (3) We solve the problem with a **non-standard algorithm**, and (4) We present managerial insights by comparing two strategic decisions which result in different versions of the problem.

The remainder of this paper is organized as follows. In Section 2, we position the problem within the related literature. In Section 3, we **formally describe the problem and we present mathematical models for its non-flexible and flexible versions**. Section 4 describes our VNS

heuristic, and in Section 5, we present our results. Finally, Section 6 offers some concluding remarks.

2. Positioning within the related literature

The problem under study belongs to the general class of pickup and delivery problems (Berbeglia et al., 2007; Parragh et al., 2008; Battarra et al., 2014), and is akin to those defined in the context of skip pickups and deliveries introduced by De Meulemeester et al. (1997), and also known as the roll-on-roll-off vehicle routing problem (Bodin et al., 2000) or as the combination truck routing problem (Li et al., 2016). The skips are loaded onto tractors to be moved between locations.

Most studies consider the unit-capacity version of the problem in which there are typically four types of elementary routes which can be combined into trips to form workdays for the tractor: (1) move an empty skip between a customer location and a depot; (2) move a full skip from a customer location to a disposal site; (3) move an empty skip from a disposal site to a depot; (4) move an empty skip from a disposal facility to a customer location. Several versions of the problem have been defined, such as considering more than one disposal site (Aringhieri et al., 2004; Baldacci et al., 2006; Benjamin and Beasley, 2010), imposing time windows (Benjamin and Beasley, 2010; Wy et al., 2013; Li et al., 2018), route duration constraints (Aringhieri et al., 2018), a driver rest period (Benjamin and Beasley, 2010), using different routing costs for the tractors, depending on whether they carry a full or an empty skip (Li et al., 2018), or minimizing the number of tractors (Rabbani et al., 2016).

Our problem is structurally different in that the tractor can carry two skips at a time, which yields the possibility of also performing elementary routes between two disposal facilities, as well as several additional ways of combining the elementary routes just described. The problems treated by De Meulemeester et al. (1997), Bodin et al. (2000), and Li et al. (2016) are not a special case of our problem, and conversely. Fig. 3 depicts the possible actions that can occur in a solution. We also consider the case of multiple disposal facilities which are associated to the content of the skip, and we impose a time limit on the working day of a tractor.

De Meulemeester et al. (1997) solved their problem exactly by branch-and-bound and approximately by means of two simple heuristics which yielded near-optimal solutions. Bodin et al. (2000) computed two lower bounds and developed four heuristics based on the solution of a set covering problem, on the combination of dynamic programming and the solution of a bin packing problem, on standard route improvement procedures such as 2-opt, 3-opt and other exchange mechanisms, and on the savings algorithm (Clarke and Wright, 1964). Aringhieri et al. (2004) proposed a heuristic based on a graph representation of an asymmetric vehicle routing problem. Archetti and Speranza (2004) developed a construction and improvement heuristic for the multi-disposal sites case and later applied it to the solution of a waste collection problem in the county of Brescia in the North of Italy (Archetti and Speranza, 2005). More recently, several authors have developed metaheuristics for versions of the problem with time windows (Benjamin and Beasley, 2010; Wy et al., 2013), and without time windows (Wy and Kim, 2013; Derigs et al., 2013; Hauge et al., 2014; Rabbani et al., 2016). The heuristic of Hauge et al. (2014) and that of Raucq et al. (2019) both make use of column generation. Li et al. (2018) apply Benders decomposition to solve the problem.

Only Archetti et al. (2005) and Raucq et al. (2019) considered the case where the tractor can carry more than one skip. In Archetti et al. (2005), the problem is defined on a graph where the depot and the unique disposal site coincide. These authors showed that if the tractor capacity is two, then the problem can be solved in polynomial time; if it is larger than two, then the problem is NP-hard.

We refer the reader to De Meulemeester et al. (1997), Archetti and Speranza (2005), Aringhieri et al. (2018), and Raucq et al. (2019) for real applications and case studies, and to Han and Ponce-Cueto

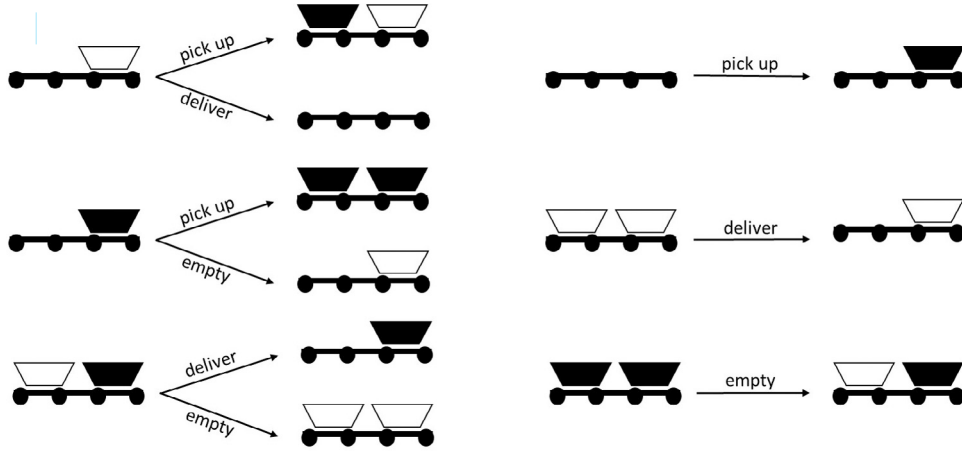


Fig. 3. The possible actions depend on the location and on the state of the tractor upon arrival. The figure shows each of the six possible states of the tractor to the left in each of the six illustrations. For each of them, the possible actions are indicated by arrows, and the right hand sides show the resulting states. White skips indicate empty skips and black skips are full. For example, in the top left figure, a truck carrying a single empty skip can either pick up a full skip or drop the empty skip.

(2015), Li et al. (2016), and Lu et al. (2017) for generic papers and surveys.

Compared with this rich body of literature, our paper constitutes one of the rare contributions to consider the case where the tractor can carry two skips. It is applied to a new problem and is the first to offer a practical solution methodology for this highly complex case.

3. Problem description and mathematical models

In this section, we describe the assumptions underlying the model and we define our problem formally. To this end, we define L as the set of locations, consisting of both pickup and delivery locations, and locations for treatment facilities. For each pair of locations, i and j , we use c_{ij} and t_{ij} to denote the distance and the travel time from i to j . We will use the terms distance and cost interchangeably in this paper. Let $R = \{r = 1, \dots, n\}$ be the set of requests. Each request r consists of a full skip which must be collected at its pickup location $\alpha(r) \in L$, transported to its treatment location $\beta(r) \in L$ where it is emptied, and finally, dropped off at its delivery location $\omega(r) \in L$. Furthermore, let F be the set of skip types, e.g., small or large, with or without cover, and let $\phi(r) \in F$ be the skip type of request $r \in R$.

We consider two versions of the problem. In the version with fixed return, $\omega(r)$ is set equal to $\alpha(r)$, i.e., each skip must return to its origin. In contrast, in the version with flexible return, $\omega(r)$ is a variable, and we require that for each location $l \in L$ and each skip type $f \in F$, we have $|\{r \in R : \alpha(r) = l, \phi(r) = f\}| = |\{r \in R : \omega(r) = l, \phi(r) = f\}|$, i.e., in the latter version, we require that the ending balance of each skip type be equal to the starting balance. In other words, for each location l and each skip type f , the number of full skips of type f picked up at l must equal the number of empty skips of type f delivered to l .

To perform these transports, are a sufficiently large set K of tractors available. Each tractor can carry two skips at a time, which can be loaded, emptied, or unloaded in any order. The tractors perform open routes, i.e., their route starts at the first pickup and is completed when the last skip is delivered. Each route is a sequence of skip pickups, emptying operations at the treatment facilities, and deliveries of empty skips. Each route starts at time zero, and must be completed within a fixed time limit T , and the time consists of driving as well as fixed operations times for pick-up (s^p), emptying (s^e), and delivery (s^d) of skips.

The aim of the problem is to identify a set of routes for the tractors such that all requests are serviced, the time duration limits are respected, and the total cost is minimized, where the cost is composed of transport cost as well as a fixed cost $\bar{c}$ for each tractor used in the solution. Compared with the traditional pickup and delivery problem,

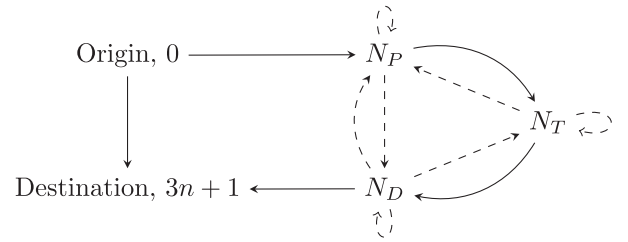


Fig. 4. Illustration of the set of arcs A in the graph underlying the mathematical formulation (1)–(18). The figure shows the connections between the different sets of nodes for the problem with fixed return. The solid arcs represent the fact that all arcs between the sets are present, while the dashed arcs indicate that arcs only exist between nodes that represent different requests.

where pairs of activities (pickups and deliveries) are scheduled, in this problem, triples of activities (pickups, emptying, and deliveries) must be scheduled, which significantly complicates the problem.

Both versions of the problem are NP-hard as they incorporated the Traveling Salesperson Problem as a special case. However, as is evident from our computational experiments in Section 5, the problem with flexible return is harder in practice than the problem with fixed return due to the larger solution space.

We now present mathematical models for the two versions of the problem.

3.1. Mathematical model for the problem with fixed return

We model the problem with fixed return of skips on a directed graph $G(N, A)$, where the node set N consists of three subsets, N_P , N_T , and N_D , each containing n nodes, as well as dummy origin and destination nodes indexed by 0 and $3n+1$, respectively. We use d to denote the dummy destination node, i.e., $d = 3n+1$. These three subsets are defined as follows: For each request i , node $i \in N_P$ represents the pickup of the full skip, node $n+i \in N_T$ represents the emptying operation at the treatment facility, and node $2n+i \in N_D$ represents the operation of delivering an empty skip to the origin of request i .

The set A of arcs is depicted in Fig. 4 and detailed in the following. From the dummy origin node 0 we add an arc $(0, j)$ to every node $j \in N_P$ representing the use of an additional tractor and the fact that the first operation must be the pickup of a full skip. We also add an arc $(0, d)$ from the dummy origin to the dummy destination representing the option of not using all tractors. For every node $i \in N_P$, we add an arc (i, j) to every node $j \in N_P, j \neq i$ as well as every node $j \in N_T$.

and $j \in N_D, j \neq i + 2n$ representing the fact that after picking up a full skip, it is possible to pick up another full skip, empty a skip, or deliver an empty skip. For every node $i \in N_T$, we add an arc (i, j) to every node $j \in N_T, j \neq i$ and to every node $j \in N_P, j \neq i - n$ and to every node $j \in N_D$ representing the fact that after emptying a skip, the tractor can either empty another skip, pick up a full skip different from the one just emptied, or deliver an empty skip. For every node $i \in N_D$, we add an arc (i, j) to every node $j \in N_D, j \neq i$, to every node $j \in N_P, j \neq i - 2n$, and to every node $j \in N_T, j \neq i - n$ representing the fact that after delivering an empty skip, the tractor can either deliver another empty skip, pick up a full skip different from the one just delivered, or empty a skip different from the one just delivered. Finally, we add arcs (i, d) from every node $i \in N_D$ to the dummy destination depot d , representing the completion of the tour. In Fig. 4, the solid arcs represent the fact that all arcs between the sets are present, while the dashed arcs indicate that arcs only exist between nodes that represent different requests.

For all arcs (i, j) , c_{ij} represents the travel cost between the locations in L corresponding to i and j , and t_{ij} represents the travel time between them. We set $t_{0j} = 0, \forall j \in N_P$ as no time consumption is associated with the start of a route. We set $c_{id} = t_{id} = 0$ for all nodes $i \in N_D$, representing the fact that we are working with open routes. Finally, we set $c_{0d} = t_{0d} = 0$ as neither cost nor time consumption is associated with an unused tractor. Define $\bar{s}_i = s_i + s_{n+i} + s_{2n+i} \forall i \in N_P$, $\bar{s}_i = s_i + s_{n+i} \forall i \in N_T$, and $\bar{s}_i = s_i \forall i \in N_D$. We associate the fixed service time s_i with every node by setting $s_i = s^p$ for $i \in N_P$, $s_i = s^e$ for $i \in N_T$, $s_i = s^d$ for $i \in N_D$, and $s_0 = s_d = 0$. We now present a two-index model for the problem with fixed return of empty skips. To control the flow of the tractors, we define the following integer variables for all arcs $(i, j) \in A$:

x_{ij} = number of times the arc (i, j) is used.

For arcs in $A \setminus \{(0, d)\}$, x_{ij} will take a binary value representing whether the arc is used or not, whereas the value of x_{0d} represents the number of unused tractors.

To control time, we define the following variables for every node $i \in N$

B_i = time when a tractor arrives at node $i \in N$.

We fix $B_0 = 0$.

To keep track of skips on the tractor and control the tractor load, we define the following parameters for all nodes $i \in N$ and for $h \in H = \{1, \dots, 2n\}$ to represent pickup and delivery, where the first n values of h represent full skips and the remaining values represent empty skips. The parameters take the following values:

$$q_i^h = \begin{cases} 1 & i \in N_P \cup N_T, h = i, \\ -1 & i \in N_T \cup N_D, h = i - n, \\ 0 & \text{otherwise.} \end{cases}$$

For a given request r , this gives $q_i^h = 1$ for $h = r$ in node, i representing the pickup of r . In the node i representing the treatment of r , we have $q_i^h = -1$ for $h = r$ and $q_i^h = 1$ for $h = r + n$, representing delivering the full skip and immediately picking up the corresponding empty skip. For node i representing the delivery of the empty skip, we have $q_i^h = -1$ for $h = r + n$. All other values of q_i^h are zero.

Finally, we define the following continuous variables for all nodes $i \in N$ and for $h \in H$:

Q_i^h = fraction of skip h on the tractor when it leaves node i

$$\forall i \in N, \forall h \in H.$$

Note that even though these variables are continuous, the constraints of the model will force them to take binary values. To ease notation, we also define Q_i^h for nodes 0 and d , and fix $Q_0^h = Q_d^h = 0$ for all $h \in H$. Furthermore, we fix $Q_i^h = 1$ for nodes $i \in N_P \cup N_T, i = h$, and $Q_i^h = 0$ for nodes $i \in N_T \cup N_D, i = h + n$ since these combinations represent pickup, empty, or delivery operations of particular skips.

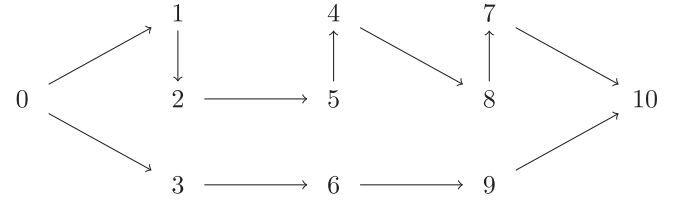


Fig. 5. Example of fixed return with three requests and two routes.

Table 1

Values of the Q_i^h -variables corresponding to the routes in Fig. 5.

		i										
		0	1	2	3	4	5	6	7	8	9	10
h full	1	0	1	1	0	0	1	0	0	0	0	0
	2	0	0	1	0	0	0	0	0	0	0	0
	3	0	0	0	1	0	0	0	0	0	0	0
h empty	4	0	0	0	0	1	0	0	0	1	0	0
	5	0	0	0	0	1	1	0	0	0	0	0
	6	0	0	0	0	0	0	1	0	0	0	0

In order to obtain a better understanding of the Q_i^h variables, consider the example in Fig. 5 with three requests and two routes. The values of Q_i^h corresponding to these routes are given in Table 1 with one column for each node i and a row for each $h \in H$, where the first three correspond to full skips and the latter three correspond to empty skips.

For node 0, the Q_i^h -values are all fixed to zero. Consider the top route. The tractor moves to node 1 to pick up the first skip. This causes $Q_1^1 = 1$ because skip 1 is now full on the tractor. The other Q_i^h -values in node 1 are still zero. The tractor then proceeds to pick up skip 2, which causes $Q_2^2 = 1$. Skip 1 is still on the tractor, so $Q_2^1 = 1$ while $Q_i^h = 0$ for the remaining values of h . The tractor now goes to node 5 to empty skip 2. This causes $Q_5^2 = 0$ and $Q_5^5 = 1$ because the tractor no longer holds the full skip 2 ($h = 2$), but instead holds the empty skip ($h = 5$). $Q_5^1 = 1$ because the full skip 1 is still on the tractor, for all other Q_i^h -variables are zero in node 5. Proceeding to node 4 to empty skip 1 causes $Q_4^1 = 0$ and $Q_4^4 = 1$ using a similar argument, and $Q_4^5 = 1$ because the empty skip 2 is still on the tractor. Continuing to node 8 to deliver skip 2 causes $Q_8^5 = 0$, but we still have the empty skip 1 on the tractor, so $Q_8^4 = 1$, while all other values are zero. Finally, traveling to node 7 to deliver skip 1 causes $Q_7^4 = 0$. The tractor is now empty and all values are zero, and it can finalize at the depot.

The second route controls the values of variables with $h = 3$ (full skip 3) and $h = 6$ (empty skip 3) in a similar manner. However, one issue still needs clarification, namely the lack of vehicle index. Consider for example node 1. the question is, how can we be sure that $Q_1^3 = 0$? Firstly, we have a constraint stating that only one tractor can enter each node (constraint (5)), so we know that tractor 2 will not enter node 1. Secondly, by definition, $q_1^3 = 0$. Therefore, by constraints (12) and (13) below, Q_1^3 can take the value zero. On the other hand, if Q_1^3 were to be one, then these two constraints would force $Q_i^3 = 1$ for all i 's on the first route because the x_{ij} -variables on that route are 1. Eventually, this would force $Q_{10}^3 = 1$ (the end depot), which conflicts with the fixing of $Q_{10}^h = 0 \forall h \in H$. Hence, the only feasible value for Q_1^3 is zero.

With this clarification of the Q_i^h -variables, we can state the model as:

$$\text{minimize} \quad \sum_{(i,j) \in A} c_{ij} x_{ij} + \sum_{j \in N_P} \bar{c} x_{0j} \quad (1)$$

$$\text{subject to} \quad \sum_{j \in N_P \cup \{d\}} x_{0j} = |K| \quad (2)$$

$$\sum_{i \in N_D \cup \{0\}} x_{id} = |K| \quad (3)$$

$$\sum_{j:(i,j) \in A} x_{ij} = 1 \quad i \in N \setminus \{0, d\} \quad (4)$$

$$\sum_{j:(j,i) \in A} x_{ji} = 1 \quad i \in N \setminus \{0, d\} \quad (5)$$

$$x_{ij} + x_{ji} \leq 1 \quad i, j \in N : (i, j), (j, i) \in A \quad (6)$$

$$B_0 = 0 \quad (7)$$

$$B_i \leq B_{n+i} \quad i \in N_P \cup N_T \quad (8)$$

$$B_i + s_i + t_{ij} - B_j \leq (1 - x_{ij})M \quad (i, j) \in A, j \neq d \quad (9)$$

$$Q_0^h = 0 \quad h \in H \quad (10)$$

$$Q_d^h = 0 \quad h \in H \quad (11)$$

$$Q_j^h \geq Q_i^h + q_j^h - (1 - x_{ij}) \quad (i, j) \in A \setminus \{(0, d)\}, h \in H \quad (12)$$

$$Q_j^h \leq Q_i^h + q_j^h + (1 - x_{ij}) \quad (i, j) \in A \setminus \{(0, d)\}, h \in H \quad (13)$$

$$0 \leq \sum_{h \in H} Q_i^h \leq 2 \quad i \in N \setminus \{0, d\} \quad (14)$$

$$0 \leq Q_i^h \leq 1 \quad i \in N, h \in H \quad (15)$$

$$0 \leq B_i \leq T - \bar{s}_i \quad i \in N \setminus \{0, d\} \quad (16)$$

$$x_{ij} \in \{0, 1\} \quad (i, j) \in A \setminus \{(0, d)\} \quad (17)$$

$$x_{0d} \geq 0 \text{ and integer.} \quad (18)$$

The objective function (1) minimizes the sum of the transport costs and the fixed cost for using the tractors. Constraints (2) and (3) ensure that all tractors leave the dummy origin and enter the dummy destination, respectively. Constraints (4) and (5) ensure that each non-dummy node is both entered and exited exactly once, thereby enforcing route continuity and ensuring that each skip is picked up, emptied, and delivered. Constraints (6) ensure that, of the two arcs connecting a pair of nodes, only one is used, thereby preventing two-node cycles. Constraints (7), (8), and (9) control time. In constraint (9), it is sufficient to use $M = T + \max_{i \in N} \{s_i\} + \max_{(i,j) \in A} \{t_{ij}\}$. Constraints (10)–(13) control the flow of skips. Constraints (12) and (13) are naturally constructed using a Big- M coefficient, but due to the one-to-one correspondence between q_i^h and Q_i^h for fixed h , we can use $M = 1$, thus simplifying the constraints to the above. Constraints (14) are the capacity constraints for the tractors. Finally, constraints (15)–(18) define the domains of the variables. Note that, due to constraints (9), constraints (6) are not necessary for the feasibility of the model, and must be viewed as valid inequalities.

3.2. Mathematical model for the problem with flexible return

For flexible returns, we use the same graph as for the fixed return case with the addition of the following arcs: For all nodes $i \in N_P$, we add arcs $(i, i + 2n)$ representing the fact that node $i + 2n$ may now correspond to a different skip than the one associated with node i . For every node $i \in N_D$, we add arcs $(i, i - 2n)$ and $(i, i - n)$, again due to the added flexibility in this version of the problem. We use the same x_{ij} variables to control tractor flow and the same B_i variables to control time. We fix $B_0 = 0$.

We redefine the load parameter as follows. For every node $i \in N$ and every request $r \in R$, we define

$$q_i^r = \begin{cases} 1 & i \in N_P, r = i, \\ -1 & i \in N_T, r = i - n, \\ 0 & \text{otherwise.} \end{cases}$$

Thus, the q_i^r parameters act exactly as the portion of the q_i^h parameters from Section 3.1 which corresponded to the full skips. We define the following continuous variables for all nodes $i \in N$ and all requests $r \in R$

$$Q_i^r = \begin{cases} \text{fraction of full skip of request } r \text{ on the tractor} \\ \text{when it leaves node } i \quad \forall i \in N, \forall r \in R. \end{cases}$$

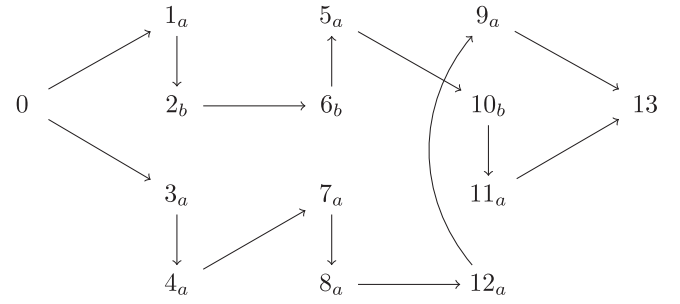


Fig. 6. Example of flexible return with three requests, two skip types, and two routes.

Table 2

Values of the Q_i^r and U_i^f variables corresponding to the routes in Fig. 6.

		i													
		0	1	2	3	4	5	6	7	8	9	10	11	12	13
h full	1	0	1	1	0	0	0	1	0	0	0	0	0	0	0
	2	0	0	1	0	0	0	0	0	0	0	0	0	0	0
	3	0	0	0	1	1	0	0	0	0	0	0	0	0	0
	4	0	0	0	0	1	0	0	1	0	0	0	0	0	0
f empty	a	0	0	0	0	0	1	0	1	2	0	1	0	1	0
	b	0	0	0	0	0	1	1	0	0	0	0	0	0	0

Jointly, the parameter q_i^r and the variables Q_i^r are used to control the flow of full skips. Similarly, we define the following parameter for all nodes $i \in N$ and all skip types $f \in F$:

$$u_i^f = \begin{cases} 1 & i \in N_T, \phi(i - n) = f, \\ -1 & i \in N_D, \phi(i - 2n) = f, \\ 0 & \text{otherwise.} \end{cases}$$

We define the following continuous variables for all nodes $i \in N$ and all skip types $f \in F$

$$U_i^f = \begin{cases} \text{fraction of empty skips of type } f \text{ on the tractor} \\ \text{when it leaves node } i \quad \forall i \in N, \forall f \in F. \end{cases}$$

Jointly, the parameter u_i^f and the variables U_i^f are used to control the flow of empty skips where only the type is relevant. Whenever a skip is emptied, and the negative q_i^r parameter ensures that the Q_i^r variable is decreased by one, the positive value of the u_i^f parameter ensures that the U_i^f variable is increased by one for the relevant skip type f . Similarly, when the empty skip is delivered, the negative value of the u_i^f parameter ensures that the U_i^f variable is decreased by one for the relevant skip type f . We can fix $Q_0^r = Q_d^r = 0$ for all $h \in R$, fix $Q_i^r = 1$ for nodes $i \in N_P, i = r$, and fix $Q_i^r = 0$ for nodes $i \in N_T, i = r + n$. Finally, we can fix $U_0^f = U_d^f = 0$ for all skip types $f \in F$.

In order to obtain a better understanding of the U_i^f variables, consider the example in Fig. 6 with four requests and two routes. We assume that there are two skip types, with skips 1, 3, and 4 being of type a and skip 2 being of type b . For ease of reading, we add the skip type as subscript of the nodes in the figure. The values of Q_i^r and U_i^f corresponding to these routes are given in Table 2 with one column for each node i . The first $n = 4$ rows give the values of Q_i^r (full skips) and the last $|F|$ rows show the values of U_i^f (empty skips of type $f \in \{a, b\}$).

Consider first the top route. The beginning of the route is similar to the example for fixed return, but when the tractor reaches node 6, the situation changes. Here, we empty skip 2, which causes $Q_6^2 = 0$, and because skip 2 is of type b , we now have $U_6^b = 1$. Proceeding to empty skip 1 in node 5, causes $Q_5^1 = 0$, and because skip 1 is of type a , we now have $U_5^a = 1$. We still have the empty skip 2 on the tractor, so $U_5^b = 1$. Hence, in node 5, the tractor carries an empty skip of each type. In node 10, the tractor delivers the skip of type b that was picked up in node 2, causing $U_{10}^b = 0$, while still carrying the skip of type a

(picked up at node 1), so $U_{10}^a = 1$. The tractor now goes to node 11 (the location where the second tractor picked up the full skip 3) and delivers the skip of type a , causing $U_{11}^a = 0$. Note that this is allowed due to the flexible return. Now, consider the second route. It starts by picking up skips 3 and 4 and then goes to node 7 to empty skip 3. This causes $Q_7^3 = 0$ and $U_7^a = 1$ because skip 3 is of type a . In node 7, the tractor still carries the full skip 4, so $Q_7^4 = 1$. Proceeding to node 8 to empty skip 4, causes $Q_8^4 = 0$. Skip 4 is of type a and because the tractor is already carrying one empty skip of type a , we now have $U_8^a = 2$. The tractor now delivers one skip of type a at node 12, causing U_{12}^a to drop to 1. The remaining skip of type a is delivered in node 9, i.e. at the location where skip 1 was picked up, resulting in complete balance of both skip types.

The flexible return problem can now be modeled as follows:

$$\text{minimize } \sum_{(i,j) \in A} c_{ij} x_{ij} + \sum_{j \in N_p} \bar{c} x_{0j} \quad (19)$$

$$\text{subject to } (2)-(6), (17), (18) \quad (20)$$

$$(7), (9), \text{ and } (16) \quad (21)$$

$$B_i \leq B_{n+i} \quad i \in N_p \quad (22)$$

$$0 \leq \sum_{r \in R} Q_i^r + \sum_{f \in F} U_i^f \leq 2 \quad i \in N \setminus \{0, d\} \quad (23)$$

$$Q_j^r \geq Q_i^r + q_j^r - (1 - x_{ij}) \quad (i, j) \in A \setminus \{(0, d)\}, r \in R \quad (24)$$

$$Q_j^r \leq Q_i^r + q_j^r + (1 - x_{ij}) \quad (i, j) \in A \setminus \{(0, d)\}, r \in R \quad (25)$$

$$Q_0^r = 0 \quad r \in R \quad (26)$$

$$Q_d^r = 0 \quad r \in R \quad (27)$$

$$0 \leq Q_i^r \leq 1 \quad i \in N, r \in R \quad (28)$$

$$U_j^f \geq U_i^f + u_j^f - M(1 - x_{ij}) \quad (i, j) \in A \setminus \{(0, d)\}, f \in F \quad (29)$$

$$U_j^f \leq U_i^f + u_j^f + M(1 - x_{ij}) \quad (i, j) \in A \setminus \{(0, d)\}, f \in F \quad (30)$$

$$U_0^f = 0 \quad f \in F \quad (31)$$

$$U_d^f = 0 \quad f \in F \quad (32)$$

$$0 \leq U_i^f \leq 2 \quad i \in N, f \in F. \quad (33)$$

In this model, constraints (20) control the flow of tractors and (21)–(22) control time. We replace constraints (10)–(15) from the fixed return model with constraints (23)–(33), where (23) ensures that the tractor capacity is respected, (24)–(28) controls the full skips and (29)–(33) controls the empty skips. The value of the Big- M in constraints (24) and (25) is one by the same argument as in the fixed return model, and in constraints (29) and (30), Big- M can be set to three.

3.3. Valid inequalities

In order to tighten the models, we have added a number of valid inequalities: (34)–(36) are new and specific to our problem, whereas (37)–(39) are adapted from Cordeau (2006). These are presented in this section.

Because the tractor can only carry two skips simultaneously, it is possible to pick up at most two skips in sequence. Furthermore, it is not possible to deliver an empty skip immediately after picking up two skips, or to empty a skip immediately before picking up two. This is illustrated in Fig. 7 and constitutes the first two sets of constraints in (34). Similarly, it is not possible to empty more than two skips in a row, or to deliver more than two in a row, as well as performing certain specific actions before and after such two actions. These constitute the remaining constraints in (34), which are valid for both models.

$$\sum_{l \in N_p, l \neq i} x_{li} + \sum_{j \in N_p \cup N_D, j \neq i} x_{ij} \leq 1 \quad i \in N_p \quad (34a)$$

$$\sum_{l \in N_p \cup N_T, l \neq i} x_{li} + \sum_{j \in N_p, j \neq i} x_{ij} \leq 1 \quad i \in N_p \quad (34b)$$

$$\sum_{l \in N_T, l \neq i} x_{li} + \sum_{j \in N_p \cup N_T, j \neq i} x_{ij} \leq 1 \quad i \in N_T \quad (34c)$$

$$\sum_{l \in N_T \cup N_D, l \neq i} x_{li} + \sum_{j \in N_T, j \neq i} x_{ij} \leq 1 \quad i \in N_T \quad (34d)$$

$$\sum_{l \in N_D, l \neq i} x_{li} + \sum_{j \in N_T \cup N_D, j \neq i} x_{ij} \leq 1 \quad i \in N_D \quad (34e)$$

$$\sum_{l \in N_D \cup N_p, l \neq i} x_{li} + \sum_{j \in N_D, j \neq i} x_{ij} \leq 1 \quad i \in N_D. \quad (34f)$$

From capacitated routing problems, it is well known that based on the demand of a subset of nodes, a lower bound on the number of tractors entering that subset can be computed. Following this logic and the fact that a tractor can carry at most two skips, we add the following six constraints to both models:

$$\sum_{i \in N_p} \sum_{j \in N_T \cup N_D \cup \{0\}} x_{ji} \geq \lceil n/2 \rceil \quad (35a)$$

$$\sum_{i \in N_p} \sum_{j \in N_T \cup N_D} x_{ij} \geq \lceil n/2 \rceil \quad (35b)$$

$$\sum_{i \in N_T} \sum_{j \in N_p \cup N_D} x_{ji} \geq \lceil n/2 \rceil \quad (35c)$$

$$\sum_{i \in N_T} \sum_{j \in N_p \cup N_D} x_{ij} \geq \lceil n/2 \rceil \quad (35d)$$

$$\sum_{i \in N_D} \sum_{j \in N_p \cup N_T} x_{ji} \geq \lceil n/2 \rceil \quad (35e)$$

$$\sum_{i \in N_D} \sum_{j \in N_p \cup N_T \cup \{d\}} x_{ij} \geq \lceil n/2 \rceil. \quad (35f)$$

We have added similar constraints for nodes corresponding to a specific action at the same physical location. For this purpose, we use the short notation N_{p_l} for the set of nodes corresponding to the pick up of a container in location l , i.e., the set $\{i \in N_p | \alpha(i) = l\}$. Similarly, we use the short notations N_{T_l} and N_{D_l} for the set of nodes corresponding to emptying and delivery at location l . This leads to constraints (36), which are added to both models:

$$\sum_{i \in N_{p_l}} \sum_{j \in \{0\} \cup (N_p \setminus N_{p_l}) \cup N_T \cup N_D} x_{ji} \geq \lceil |N_{p_l}|/2 \rceil \quad l \in L \quad (36a)$$

$$\sum_{i \in N_{T_l}} \sum_{j \in N_p \cup (N_T \setminus N_{T_l}) \cup N_D} x_{ji} \geq \lceil |N_{T_l}|/2 \rceil \quad l \in L \quad (36b)$$

$$\sum_{i \in N_{D_l}} \sum_{j \in N_p \cup N_T \cup (N_D \setminus N_{D_l})} x_{ji} \geq \lceil |N_{D_l}|/2 \rceil \quad l \in L. \quad (36c)$$

Since the problem is a routing problem, traditional subtour elimination constraints can be applied to it. Cordeau (2006) showed how these can be lifted for pickup and delivery problems. For pairs of nodes $i, j \in N_T$, and triplets $i, j, h \in N_T$, we have added the lifted subtour elimination constraints of Cordeau (2006) to the models. For pairs, we have added the constraints (37), where the first set applies to both models, whereas the second only applies to the fixed return model. The extension to triplets is straightforward resulting in a right-hand side of 2. For the fixed return model, the constraints can be lifted even further based on the specific structure of the problem for pairs and triplets in N_p and N_D . This is illustrated in Fig. 8 for the pairs $i, j \in N_p$ and in Fig. 9 for the pairs $i, j \in N_D$. This provides the two sets of constraints in (38) for pairs of nodes. Again the extension to triplets is straightforward. Inequalities (38) do not apply to the flexible case. Instead, we add (39).

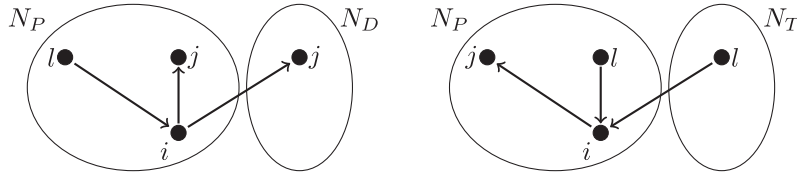
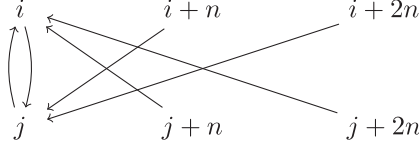
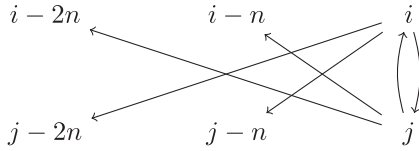
$$x_{ij} + x_{i,j-n} + x_{ji} + x_{j,i-n} \leq 1 \quad i, j \in N_T, i \neq j \quad (37a)$$

$$x_{ij} + x_{i+n,j} + x_{ji} + x_{j+n,i} \leq 1 \quad i, j \in N_T, i \neq j; \quad (37b)$$

$$x_{ij} + x_{i+n,j} + x_{i+2n,j} + x_{ji} + x_{j+n,i} + x_{j+2n,i} \leq 1 \quad i, j \in N_p, i \neq j \quad (38a)$$

$$x_{ij} + x_{i,j-n} + x_{i,j-2n} + x_{ji} + x_{j,i-n} + x_{j,i-2n} \leq 1 \quad i, j \in N_D, i \neq j; \quad (38b)$$

$$x_{ij} + x_{i+n,j} + x_{ji} + x_{j+n,i} \leq 1 \quad i, j \in N_p, i \neq j. \quad (39)$$

Fig. 7. Illustration of constraint (34) for $i \in N_P$.Fig. 8. Illustration of the lifted subtour elimination constraints (38a) for $i, j \in N_P$.Fig. 9. Illustration of the lifted subtour elimination constraints (38b) for $i, j \in N_D$.

We have also added a number of symmetry-breaking constraints. The first three sets (40) relate to the order of the requests, when two similar actions (for instance, two pickup actions) are performed in a row, and apply to both models. The sets (41) define the order of different actions performed in sequence at the same location, and are also added to both models. To break symmetry further, we add constraints (42) to the model with fixed return and (43) to the flexible return model:

$$x_{ij} = 0 \quad i, j \in N_P, \alpha(i) = \alpha(j), i > j \quad (40a)$$

$$x_{ij} = 0 \quad i, j \in N_T, \beta(i) = \beta(j), i > j \quad (40b)$$

$$x_{ij} = 0 \quad i, j \in N_D, \omega(i) = \omega(j), i > j; \quad (40c)$$

$$x_{ij} = 0 \quad i \in N_P, j \in N_D, \alpha(i) = \omega(j) \quad (41a)$$

$$x_{ij} = 0 \quad i \in N_T, j \in N_P, \beta(i) = \alpha(j) \quad (41b)$$

$$x_{ij} = 0 \quad i \in N_D, j \in N_T, \omega(i) = \beta(j); \quad (41c)$$

$$B_i \leq B_j \quad \forall i, j \in N_P, i < j, \alpha(i) = \alpha(j), \beta(i) = \beta(j); \quad (42)$$

$$B_i \leq B_j \quad \forall i, j \in N_P, i < j, \alpha(i) = \alpha(j), \beta(i) = \beta(j), \phi(i) = \phi(j). \quad (43)$$

By considering travel and service times, we can compute a lower bound on the number of tractors needed to service all requests as $k_{\min} = \lceil (\sum_{r \in R} t_{\alpha(r)\beta(r)} + \sum_{i \in N} s_i) / T \rceil$ in the case with fixed return, and as $k_{\min} = \lceil ((\sum_{r \in R} t_{\alpha(r)\beta(r)} + \sum_{r \in R} (\min_{r' \in R, \phi(r)=\phi(r')} t_{\beta(r)\omega(r')}) / 2 + \sum_{i \in N} s_i) / T \rceil$ in the case with flexible return. This bound is added to the models by applying constraint (44):

$$\sum_{j \in N_P} x_{0j} \geq k_{\min}. \quad (44)$$

4. Variable neighborhood search-inspired heuristic

In this section, we describe our VNS-inspired heuristic. Because the problem is highly constrained, we allow intermediate infeasible solutions in order to better explore the search space, and we use a simulated annealing (SA) acceptance criterion in both layers of the algorithm.

Starting from an incumbent solution x , the overall idea in a VNS algorithm is for the outer layer to shake the solution by alternating between neighborhood structures in order to obtain another solution x' . Then x' is given to a variable neighborhood descent procedure (the inner layer) which will output a solution x'' . At this point, x and x'' are compared, and it is determined which of the two should be the incumbent solution in the next iteration. Our algorithm follows this overall outline.

In the following, we first briefly describe our constructive heuristic in Section 4.1. We then present the main algorithm without allowing infeasible solutions in Section 4.2, and the corresponding inner layer in Section 4.3. Finally, we describe the modifications needed to handle infeasible solutions, thereby also introducing the penalty factor P and the working time limit $\tilde{T}$ in Section 4.4.

Due to the large number of parameters, we provide in Table 3 their values which were obtained through a thorough tuning phase. For ease of reading, this is done here rather than in Section 5.

4.1. Initial solution

Our algorithm is initialized from a feasible solution obtained by a simple greedy constructive heuristic that constructs one route at a time. The heuristic adds pairs of requests to the end of the route one by one to form patterns of the form ‘pickup, pickup, empty, empty, deliver, deliver’. In the heuristic, all skips are returned to their own origin (fixed return). For each route, the pairs are chosen in the following way. When choosing the first pair, priority is given to pairs that share both pickup location and treatment location. If no such pair exists, we choose the pair that requires to shortest possible time to complete its processing.

To identify subsequent pairs r and r' to add, we first identify the request r whose pickup location is closest to the end of the current partial route. If multiple such requests exists, we give priority to choosing one among them that shares a pickup location with another unscheduled request. Once r is chosen, we choose r' among all unserved requests as the one that, paired with r , causes the least additional processing time when added to the current partial route. This process is continued as long as pairs of requests can be added without exceeding T . At the end of the route, when no more pairs can feasibly be added, a single request is added if it is feasible to do so. This request is chosen as the one whose pickup location is closest to the end of the current partial route and which can be added to the current partial route without causing the route to exceed T .

4.2. The main algorithm

Algorithm 1 describes the outer layer of our heuristic. The main purpose of this part of the algorithm is to diversify the search. We start from a feasible solution x , obtained by the constructive heuristic described in Section 4.1 (line 1) and record it as the current best solution, x^B (line 2). The main loop (lines 6–18) controls the SA cooling scheme, using t as temperature and θ_{start} and θ_{end} as initial and ending temperature, respectively, and the outer loop (lines 3–19) controls reheating of the system for a fixed number of iterations NUMREHEAT. At each iteration, we randomly select one of the neighborhoods with a certain probability (line 7) and obtain a new solution x' , using that neighborhood (line 8). x' may exceed T , but it will respect $\tilde{T}$.

Table 3
Values of parameters tested during the tuning of the algorithm.

Parameter	Tested values	Final value
MAINALGORITHM, α	0.99, 0.995	0.995
MAINALGORITHM, θ_{start}	50, 100	100
MAINALGORITHM, θ_{end}	0.05, 0.1, 0.2	0.05
MAINALGORITHM, NumReheat	10, 20, 30, 50, 75, 100	100
MAINALGORITHM, Reheat temperature	$0.5\theta_{start}, 0.75\theta_{start}, \theta_{start}$	$0.5\theta_{start}$
MAINALGORITHM, Percent empty route, flexible return	10, 15, 20, 25, 30	15
MAINALGORITHM, Percent move, flexible return	10, 15, 20, 30, 40	15
MAINALGORITHM, Percent swap, flexible return	20, 25, 30, 35, 40, 50	30
MAINALGORITHM, Percent swap return location, flexible return	20, 30, 35, 40	40
UPDATETIMELIMIT, λ_1	5, 10, 20	5
UPDATETIMELIMIT, λ_2	5, 10, 20	10
UPDATETIMELIMIT, T_{max}	$1.5T, 2T$	$1.5T$
UPDATETIMELIMIT, Iterations between change of $\tilde{T}$	5, 10, 20	5
ONE-ROUTE-SA ($\mathcal{R}, \tilde{T}$), α	0.9, 0.95, 0.99	0.9
ONE-ROUTE-SA ($\mathcal{R}, \tilde{T}$), θ_{start}	50, 75, 100	50
ONE-ROUTE-SA ($\mathcal{R}, \tilde{T}$), θ_{end}	0.05, 0.1, 0.2	0.1
ONE-ROUTE-SA ($\mathcal{R}, \tilde{T}$), θ_{end} , after final reheat	0.05, 0.1	0.1
ONE-ROUTE-SA ($\mathcal{R}, \tilde{T}$), NumReheat	0, 1, 2, 5, 10, 20	5
UPDATEONEROUTEPENALTY, Iterations between change of P	5, 10, 15	10
UPDATEONEROUTEPENALTY, Factor for decreasing P	0.5, 0.75	0.75
UPDATEONEROUTEPENALTY, Factor for increasing P	1.1, 1.3	1.1

Depending on the neighborhood, one or several routes will be modified. Each of these routes, $\mathcal{R}(x')$ is sent to the lower level, where our ONE-ROUTE-SA procedure will modify them (line 9). The resulting solution is x'' . We then use a SA acceptance criterion to determine whether x'' should be the new incumbent (lines 10–11) and **update x^B if needed** (lines 12–13).

Algorithm 1 MAINALGORITHM

```

1:  $x \leftarrow \text{CONSTRUCTIVEHEURISTIC}$ 
2:  $x^B \leftarrow x$ 
3: for  $k = 1$  To NumReheat + 1 do
4:    $t \leftarrow \theta_{start}$ 
5:    $\tilde{T} \leftarrow T$ 
6:   while  $t > \theta_{end}$  do
7:      $\rho \leftarrow \text{GETRANDOMNEIGHBORHOOD}$ 
8:      $x' \leftarrow N_\rho(x, \tilde{T})$ 
9:      $x'' \leftarrow \text{ONE-ROUTE-SA}(\mathcal{R}(x'), \tilde{T})$   $\triangleright$  for affected routes
10:    if  $x''$  is feasible  $\wedge \text{ACCEPT}(c(x'') - c(x), t)$  then
11:       $x \leftarrow x''$ 
12:      if  $c(x) < c(x^B)$  then
13:         $x^B \leftarrow x$   $\triangleright$  Save the best
14:      end if
15:    end if
16:     $\tilde{T} \leftarrow \text{UPDATETIMELIMIT}(\tilde{T})$ 
17:     $t \leftarrow \alpha t$ 
18:  end while
19: end for
20: return  $x^B$ 

```

The outer layer of our algorithm uses four neighborhood structures. Due to the complexity of ensuring that the routes are pattern-feasible, i.e., ensuring that all activities for the single skip (i.e. the triple consisting of pickup, emptying, and delivery) are performed in the correct order, and that there are never more than two skips on the tractor simultaneously, we have chosen to always insert requests at the end of the routes. The neighborhood structures are as follows, where the numbers in parentheses are the probability of using the neighborhood structure in the case of fixed/flexible return:

1. Move a random request to the end of a random route (25%/15%).

2. Swap two random requests (from the same or different routes) (50%/30%).
3. Empty the route with the shortest duration by repeatedly removing the request that is picked up first from the route, and greedily selecting a route where it is added to the end in order to minimize the insertion cost. If this is not possible due to time limits, a new route is created for the request (25%/15%).
4. This neighborhood is only used in the case where we allow flexible return of the empty skips. It randomly selects two requests with the same skip type which are currently delivered to different locations, and it swaps the delivery location of those requests. The probability of selecting each skip type is proportional to percentage of that skip type in the problem at hand. (0%/40%).

Several other variations of these neighborhood structures were tested, but this combination was the best.

4.3. Inner layer of the algorithm

Algorithm 2 describes the ONE-ROUTE-SA that we use in the inner layer. The algorithm involves notation that is necessary for handling infeasible solutions, which are thoroughly introduced in Section 4.4. We briefly state the notation here for completeness. Given a route $\mathcal{R}$, we use $c(\mathcal{R})$ and $d(\mathcal{R})$ to denote the cost and the duration of the route, respectively, and we use $e(\mathcal{R})$ to denote the amount by which the route duration exceed T . The procedure seeks to optimize a single route $\mathcal{R}$ by applying a SA to reshuffle the activities on the route. The main loop (lines 5–18) controls the SA cooling scheme, again using t as temperature and θ_{start} and θ_{end} as initial and ending temperature, respectively. The RANDOMPATTERNSWAP neighborhood (line 6) randomly selects an activity on the route, which can either be the pickup of a full skip, an emptying operation, or the delivery of an empty skip. It then swaps the activity with the one succeeding it in $\mathcal{R}$, resulting in route $\mathcal{R}'$. If $\mathcal{R}'$ is pattern-feasible (line 7), then a SA acceptance criterion is used to determine whether $\mathcal{R}$ or $\mathcal{R}'$ should be the new incumbent (line 9). Finally, the best route so far, $\mathcal{R}^B$ is updated (lines 11–13), and the penalty factor P is adjusted (line 16). After the main loop, we use the MOVESWAPTOBACK procedure, which $|\mathcal{R}|$ times moves a request selected uniformly at random from the route to the end of it, thereby re-shuffling the route (lines 19–21). This whole process is repeated six times (NUMREHEAT+1) (lines 2–22). We use $\theta_{start} = 50, \theta_{end} = 0.1$, and $\alpha = 0.9$.

Algorithm 2 ONE-ROUTE-SA ($\mathcal{R}, \tilde{T}$)

```

1:  $\mathcal{R}^B \leftarrow \mathcal{R}$ 
2: for  $k = 1$  To NumReheat + 1 do
3:    $t \leftarrow \Theta_{start}$ 
4:    $P \leftarrow P_{start}$  ▷ Reset the penalty
5:   while  $t > \Theta_{end}$  do
6:      $\mathcal{R}' \leftarrow \text{RANDOMPATTERNSWAP}(\mathcal{R}, \tilde{T})$ 
7:     if  $\mathcal{R}'$  is pattern-feasible then
8:        $e(\mathcal{R}) = \max\{0, d(\mathcal{R}) - \tilde{T}\}$ 
9:       if  $c(\mathcal{R}') \leq c(\mathcal{R}) \vee \text{ACCEPT}(c(\mathcal{R}') - c(\mathcal{R}) + P(2e(\mathcal{R}') - e(\mathcal{R})), t)$ 
10:        then
11:           $\mathcal{R} \leftarrow \mathcal{R}'$ 
12:          if  $(e(\mathcal{R}^B) > 0 \wedge (d(\mathcal{R}) < d(\mathcal{R}^B) \vee (d(\mathcal{R}) = d(\mathcal{R}^B) \wedge c(\mathcal{R}) < c(\mathcal{R}^B)))) \vee (e(\mathcal{R}) < 0 \wedge c(\mathcal{R}) < c(\mathcal{R}^B))$  then
13:             $\mathcal{R}^B \leftarrow \mathcal{R}$ 
14:          end if
15:        end if
16:         $P \leftarrow \text{UPDATEONEROUTEPENALTY}(P)$ 
17:         $t \leftarrow \alpha t$ 
18:      end while
19:      loop  $|\mathcal{R}|$  times
20:         $\mathcal{R} \leftarrow \text{MoverandomToBack}(\mathcal{R})$ 
21:      end loop
22:    end for
23: return  $\mathcal{R}^B$ 

```

Table 4
Parameters tuned in each phase.

Ph.	Part	Parameters	#
1	MAINALG.	$\alpha, \Theta_{start}, \Theta_{end}, \text{NumReheat}, \text{Reheat temp.}$	84
2	ONE-R.SA	$\alpha, \Theta_{start}, \Theta_{end}, \Theta_{end-final}, \text{and NumReheat}$	16
3	MAINALG.	Percent for each neighborhood	37
4	UPD.TIME.L.	$\lambda_1, \lambda_2, T_{max}, \text{and it. between change of } \tilde{T}$	27
5	UPD.ONER.P.	It. between change of P , Factor for decr. or incr. P	10
6	MAINALG.	$\Theta_{end}, \text{NumReheat}$	20
	ONE-R.SA	$\Theta_{end}, \Theta_{end-final}, \text{NumReheat}$	

4.4. Infeasibility and penalty adjustment

Because we always insert requests at the end of the routes in the main algorithm, and let the ONE-ROUTE-SA rearrange the route, we need to temporarily allow the routes to exceed the time duration limits, thus allowing intermediate infeasible solutions. However, throughout the algorithm, we only allow pattern-feasible routes. To describe how infeasibilities are handled by the algorithm, we need some additional notation. We work with two route time limits. The true time limit T , and a working time limit $\tilde{T}$, where the former is used to ensure that solutions are feasible, and the latter is gradually adjusted throughout the algorithm in order to balance the search. We use a fixed parameter, T_{max} , as an upper bound on $\tilde{T}$. For each route $\mathcal{R}$, we define $c(\mathcal{R})$ as the cost of the route, $d(\mathcal{R})$ as its duration, and $e(\mathcal{R})$ as its excess duration with respect to $\tilde{T}$, i.e., $e(\mathcal{R}) = \max\{0, d(\mathcal{R}) - \tilde{T}\}$. Finally, we use $\mathcal{R}(x)$ to indicate a route of the solution x .

In the main algorithm, we allow for the routes generated by the neighborhood structures (line 8) to exceed the true duration time limit by using the working time limit $\tilde{T}$. We will then let the ONE-ROUTE-SA seek to force the length of the route to become less than T during the optimization process, and in the main algorithm we only accept to the solution if all routes are feasible with respect to the true time limit T (line 10). In UPDATETIMELIMIT, we update $\tilde{T}$ as follows. If x' has been feasible for five consecutive iterations, we set $\tilde{T} = \min\{T_{max}, \tilde{T} + \lambda_1\}$ with $\lambda_1 > 0$, i.e., we increase $\tilde{T}$ by λ_1 , but we never increase it above a fixed maximum level. On the other hand, if x' has been infeasible for

five consecutive iterations, we set $\tilde{T} = \max\{T, \tilde{T} - \lambda_2\}$, with $\lambda_2 > 0$, i.e., we decrease $\tilde{T}$ by λ_2 , but we never below the true time limit. In both cases, we reset the count of iterations. We use $T_{max} = 1.5T$, $\lambda_1 = 5$, and $\lambda_2 = 10$. Finally, the system is reheated 100 times to a temperature of $0.5\Theta_{start}$, and when it is reheated (lines 3–19), $\tilde{T}$ and all penalties are reset and we set $x = x^B$.

The aim of ONE-ROUTE-SA is to simultaneously optimize the route in terms of cost, and seek to make the route time-feasible. For this purpose, we use the penalty factor, P , in the SA acceptance criterion (line 8), where we set

$$\Delta = c(\mathcal{R}') + 2Pe(\mathcal{R}') - (c(\mathcal{R}) + Pe(\mathcal{R})).$$

The logic of this calculation is that we heavily penalize exceeding the time limit in the new route, while we also encourage leaving routes that exceed their time limit. In the determination of the current best solution (line 10), we focus on both the cost of the route and whether it exceeds the time limit. Therefore, we define a better route as one that exceeds time by less than the current $\mathcal{R}^B$ or which exceeds by the same amount (or not at all) and is cheaper. The procedure UPDATEONEROUTEPENALTY updates P as follows: If $\mathcal{R}$ has been feasible for 10 consecutive iterations, we set $P = 0.75P$, and if $\mathcal{R}$ has been infeasible for 10 consecutive iterations, we set $P = 1.1P$.

Combining the above ways to handle infeasibility, intuitively, $\tilde{T}$ controls the extent to which the routes sent to ONE-ROUTE-SA may exceed the T , and P is used as a penalty inside ONE-ROUTE-SA to help the route become feasible. However, it may very well be the case that ONE-ROUTE-SA is unable to make the route time feasible, or this may even be impossible. Therefore, we ensure, in the main algorithm, that solutions are only accepted if all routes are feasible (line 10).

5. Computational results

The algorithm was implemented in C++ in MS Visual Studio Professional 2015 and executed on an Intel Xeon CPU with 12 cores running at 3.5 GHz and 64 GBs RAM. It was executed sequentially, i.e., without taking advantage of the multiple cores.

Tuning of the parameters in Table 3 was done on 12 instances,¹ three of which are not used in the final experiments. Except from the last three, the instances sizes are in the small range of the instances used in the final experiments due to computation time. There is a total of $1.36 \cdot 10^{10}$ combinations of the parameters, which makes it impossible to test all combinations. Therefore, the parameters are tuned in six phases, where each phase contained a sufficient number of combinations for us to be confident about the values of the parameters. Table 4 provides information about the parameters tuned in each phase as well as the number of parameter combinations tested in each phase. In total, 194 parameter combinations were tested in the formal timing phase.

The algorithm was run on 80 real-life data sets provided by our industrial partners from four areas in Denmark, as illustrated in Fig. 10. Each data set lists the skips that were emptied on a given day of operation together with information of the content of the skip and its type. The place to empty the skip is based on its content, and the flexibility regarding return of skips is determined by the skip type. The values of c_{ij} and t_{ij} are obtained from MapPoint using a balanced setting where priority is given to driving on main roads, and may not always satisfy the triangle inequality, but they are symmetric. The instances contain between five and 185 requests. All instances use a fixed cost of 500 for each route and $T = 8$ hours. The value of 500 is sufficiently high to ensure that the first priority is given to minimizing the number of tractors. The data files are available upon request. The algorithm was run 10 times for each instance both with fixed and

¹ Instances A9, A13, B2, B9, B10, C2, C3, C6, D1, and three instances with 130, 173, and 180, which are not used in the experiments.

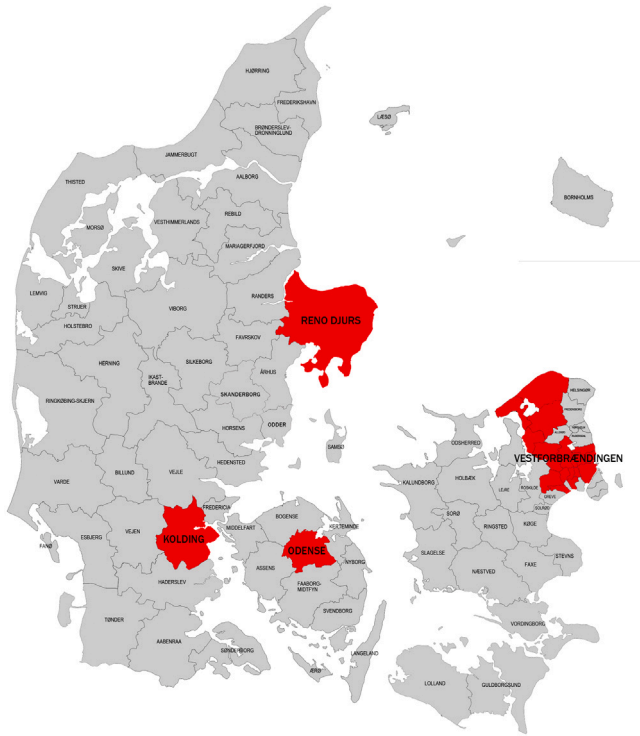


Fig. 10. The areas covered by the data.

flexible return of skips. The data sets and our solutions are available at www.optimization.dk/skips.

Tables 5 and 6 provide an overview of the results. The first four columns provide information regarding the instances: the ID number, the number of requests ($|R|$), the number of skip types ($|F|$), and the number of treatment facilities ($|TR|$). The next four columns provide the results for fixed return, while the last four are devoted to flexible return. Among these, the first column ($\bar{R}$) is the average number of routes used in the 10 runs, the second ($C_{\min}$) is the best cost obtained over the 10 runs, while the third (C_{av}) is the average cost. Note that both $C_{\min}$ and C_{av} consist of the fixed vehicle costs as well as travel costs. Finally, the fourth column (Time) provides the average run time over the 10 runs.

We first consider the stability of our algorithm across the 10 runs for each instance. Between the 160 instances (80 with and without flexibility), the highest coefficient of variation is 10.6%, and it is below 1% (5%) for 123 (154) of the 160 instances. This indicates a very stable performance of the algorithm. The runtime of the heuristic is shown in Fig. 11. It is generally quite fast: less than 30 s for up to 50 requests and it has a slow linear growth with the number of requests. The runtime is slightly smaller for flexible return than for fixed return, which is likely explained by the fact that exploring the neighborhood that changes delivery location, which is only used for these instances, requires less computational effort than the other neighborhoods.

In Table 7, we take a deeper look at flexible returns. In the table, we show the details of one of the routes in the best solution of instance C20. The route services eight requests and we show, in two layers, the request ID, the skip type, the location, the action (pickup, empty, delivery), and the tractor load after the action. It is interesting to see that only one skip (request 58) is returned to the location at which it was picked up. We see that the tractor of this route services one request with skip of type 2. The skip is picked up at location 4, but delivered to location 1. Consequently, another tractor will return a skip of type 2 to location 4 and that tractor, or another one, will pick up a skip of type 2 from location 1. This illustrates how the skip balance constraint must

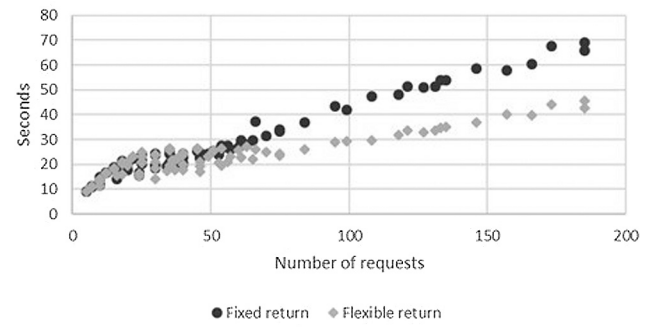


Fig. 11. Runtime of the heuristic.

hold at a global level, but not necessarily at the level of the individual route. In our example, three skips caused such an imbalance. Finally, consider the action patterns of the route. In the first part of the route, this pattern is simple, whereas in the second part of the route, the pattern is more complicated.

On average, 62.6% of the requests return to a different location from their own (not just a different skip slot at the same recycling center, but a different geographical location). The average is taken over all 80 data sets and over 10 runs of each. The results are plotted in Fig. 12 where each point corresponds to a data set, with average over the 10 runs.

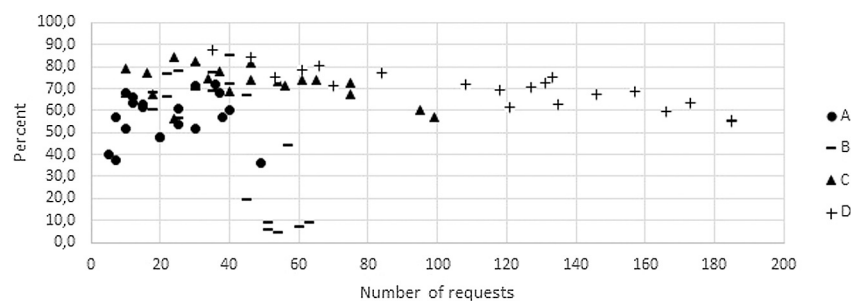
Finally, we compare the results obtained with fixed and flexible return (see Tables 5 and 6). The number of routes generally grow linearly with the number of requests. The B-instances need fewer routes per request than the other instances since the B-instances generally have smaller distances. When we compare the average number of routes over the 10 runs, we observe that in 55 of the instances, the number of routes is the same for fixed and flexible return, while in 22 instances, the average number of routes with flexible return is smaller than for fixed return. In three cases, we observe the opposite. This is due to the uncertainty of the heuristic comparison. However, the difference is only 0.2 route for these three instances. By comparing the average cost including (excluding) the fixed cost per route, we observe that the average saving obtained by allowing flexible return is 2.3% (6.1%), and we observe such cost savings in 76 (77) of the 80 instances.

We have implemented the models of Sections 3.1 and 3.2 in CPLEX and run them on the same computer with a run time limit of two hours for each instance with up to 20 requests. CPLEX is initiated from the solution obtained with one run of our heuristic. The results are shown in Table 8. For each of the two models, we show the lower bound (LB), the best feasible solution (UB), and the run time, where 'TL' indicates that the run time limit is reached. For convenience, we also repeat the best solution obtained by our heuristic (Heu). The lower bound value is marked in bold if the solution is optimal. The feasible solutions from CPLEX and our heuristic values are marked in bold if the solution is the best. From these results, we draw three conclusions: (1) The models can only solve the smallest instances to optimality. Of the 17 instances, we solved six to optimality with fixed return and three with flexible return. (2) The model with flexible return is generally harder to solve and leads to an average gap of 22.0%, compared with 11.6% for the fixed return model. These results are obtained by the models that include all the valid inequalities presented in Section 3.3. The gaps obtained without the use of these valid inequalities, but with the symmetry breaking constraints, are 31.3% and 11.8%, respectively. Further analyses of the usefulness of the valid inequalities are provided in Appendices A and B. The main conclusion is that all the valid inequalities are useful in some of the instances for the fixed return problem, and that all except (34) are useful for the flexible return problem. (3) The best solutions of our heuristic runs, each of which last less than half a minute, are at least as

Table 5

Detailed results for A and B instances. The table shows the instance ID, the number of requests ($|R|$), the number of skip types ($|F|$), the number of treatment facilities ($|TR|$), and for each of the two versions: the average number of routes used in the 10 runs ($\bar{R}$), the best cost obtained over the 10 runs ($C_{\min}$) and the average cost (C_{av}) both of which includes both the fixed vehicle costs and travel cost, as well as the average run time in seconds among the 10 runs (Time).

Instances				Fixed				Flexible return			
ID	$ R $	$ F $	$ TR $	$\bar{R}$	$C_{\min}$	C_{av}	Time	$\bar{R}$	$C_{\min}$	C_{av}	Time
A1	5	4	4	1	661	661.0	9.0	1	648	648.0	9.1
A2	7	3	3	1	645	645.0	11.0	1	625	625.0	10.9
A3	7	2	7	1	628	628.0	11.3	1	619	619.0	11.3
A4	10	2	3	1	720	720.0	13.0	1	695	695.0	12.9
A5	10	4	6	1	724	725.0	11.5	1	707	709.3	11.0
A6	12	3	4	2	1240	1240.0	16.5	2	1202	1202.2	16.3
A7	12	3	7	2	1248	1248.0	16.7	2	1211	1214.9	16.5
A8	15	4	4	2	1356	1356.0	18.1	2	1314	1314.0	17.9
A9	15	4	7	2	1253	1253.0	18.9	2	1244	1244.0	18.4
A10	20	3	4	2	1316	1316.1	20.1	2	1297	1298.1	20.9
A11	20	6	7	2	1342	1343.0	17.7	2	1289	1293.5	19.4
A12	25	4	5	3	1935	1939.0	20.1	3	1920	1925.9	20.5
A13	25	4	7	3	1929	1940.1	20.6	3	1879	1891.7	20.8
A14	30	4	6	3	1979	1988.1	19.6	3	1951	1963.6	21.1
A15	30	7	6	3	2076	2085.8	18.4	3	2077	2085.5	18.6
A16	36	5	7	4	2673	2688.1	21.5	4	2642	2657.3	21.8
A17	37	4	6	4	2679	2689.7	21.5	4	2638	2662.1	21.8
A18	38	5	6	4	2695	2706.8	22.2	4	2682	2695.1	22.5
A19	40	6	8	4	2750	2762.4	20.1	4	2702	2723.8	19.8
A20	49	5	6	4.7	2745	3100.2	24.1	5	3251	3260.2	23.2
B1	18	6	5	2	1323	1325.5	20.2	2	1299	1306.3	20.3
B2	18	5	4	2	1148	1150.8	21.3	2	1121	1128.9	20.8
B3	22	4	3	2	1188	1193.1	22.4	2	1163	1172.9	23.0
B4	22	6	5	2	1178	1181.3	22.4	2	1156	1169.2	23.0
B5	25	6	5	2	1171	1176.2	23.8	2	1166	1171.2	24.8
B6	25	5	3	2	1205	1212.9	21.5	2	1192	1196.1	23.3
B7	30	6	4	3	1761	1764.5	24.1	3	1751	1754.8	23.9
B8	30	5	5	3	1759	1765.8	24.0	3	1739	1748.2	24.0
B9	35	6	4	3	1741	1757.1	25.8	3	1737	1752.0	26.5
B10	35	6	6	3	1846	1855.0	24.2	3	1833	1846.2	25.6
B11	40	5	4	3	1812	1826.9	22.7	3	1801	1812.0	23.9
B12	40	6	6	3.7	1934	2292.6	24.1	3.9	1916	2396.0	24.7
B13	45	6	5	4	2393	2401.2	25.8	4	2378	2397.0	26.5
B14	45	4	5	4	2390	2396.5	25.4	4	2373	2390.3	25.9
B15	51	5	4	4	2407	2419.5	25.3	4	2410	2422.2	25.8
B16	51	7	5	4	2416	2423.9	25.3	4	2415	2419.1	25.4
B17	54	6	4	4	2391	2402.1	26.4	4	2390	2398.8	26.4
B18	57	6	6	4	2518	2526.3	25.9	4	2497	2508.8	23.0
B19	60	6	6	5	3161	3170.3	26.7	5	3158	3167.7	25.7
B20	63	6	5	5	3104	3115.7	27.9	5	3092	3105.2	27.0

**Fig. 12.** Percent of the skips that are returned to a location different from their origin.

good as the best feasible solution obtained by CPLEX within two hours for all but two instances, even though CPLEX is warm-started from a good heuristic solution, and is strictly better for most of the instances with flexible return. These findings support our solution approach and confirm the quality of our heuristic. In [Appendix C](#), we use a truncated version of CPLEX as a heuristic and compare it to our heuristic. The results show that our heuristic is far superior in obtaining good feasible solutions even though our heuristic runs in less than half a minute and CPLEX is allowed two hours of computation time.

6. Concluding remarks and managerial insights

We have studied a skip pickup and delivery problem based on a practical problem faced by a number of industrial partners in the waste industry. Our study is rooted in a real Danish application, but is of general applicability. The problem is to optimally transport skips between recycling centers and treatment facilities. We have considered two cases of the problem: one where the skips must be returned to their origin, and one where the return locations are unspecified. We have formulated the two versions of the problem as mixed integer linear

Table 6

Detailed results for C and D instances. The table shows the instance ID, the number of requests ($|R|$), the number of skip types ($|F|$), the number of treatment facilities ($|TR|$), and for each of the two versions: the average number of routes used in the 10 runs (R), the best cost obtained over the 10 runs ($C_{\min}$) and the average cost (C_{av}) both of which includes both the fixed vehicle costs and travel cost, as well as the average run time in seconds among the 10 runs (Time).

Instances				Fixed				Flexible return			
ID	$ R $	$ F $	$ TR $	R	$C_{\min}$	C_{av}	Time (s)	R	$C_{\min}$	C_{av}	Time (s)
C1	10	2	2	2	1 362	1 362.0	14.9	2	1 297	1 297.7	14.2
C2	10	4	5	2	1 481	1 481.0	12.6	2	1 438	1 444.0	13.2
C3	16	5	5	3	2 331	2 331.7	14.2	3	2 269	2 279.3	15.4
C4	18	4	2	3	2 034	2 034.0	16.8	2.8	1 520	1 892.3	15.9
C5	24	4	5	4	2 912	2 921.0	16.5	4	2 810	2 836.9	16.2
C6	24	5	7	4	3 123	3 153.7	15.4	4	3 035	3 045.7	15.3
C7	30	5	6	4.9	3 137	3 532.7	19.0	4	2 899	2 933.4	14.2
C8	34	5	7	5	3 823	3 848.9	19.2	5	3 568	3 590.5	17.5
C9	37	5	5	6	4 712	4 740.5	20.1	6	4 498	4 538.7	17.7
C10	40	5	5	6	4 692	4 742.2	20.3	6	4 487	4 524.8	17.7
C11	46	5	5	7	5 451	5 486.4	24.6	7	5 312	5 340.4	19.0
C12	46	5	7	7	5 218	5 253.4	22.8	6.1	4 458	4 550.2	17.2
C13	54	5	7	8	6 202	6 284.3	27.3	8	5 974	6 030.7	19.7
C14	56	5	5	8.6	6 302	6 625.3	27.5	8.3	6 147	6 349.9	21.1
C15	61	5	5	10	7 566	7 650.2	29.7	9.5	6 937	7 278.7	22.8
C16	65	5	5	10	7 569	7 637.3	29.5	9.9	7 044	7 475.3	22.2
C17	75	5	5	11.5	8 532	8 856.5	33.2	11	8 357	8 454.7	24.2
C18	75	5	6	11.1	8 540	8 643.9	34.1	11	8 329	8 432.6	23.6
C19	95	5	5	14.8	11 069	11 436.5	43.3	14.1	10 818	10 971.0	29.0
C20	99	5	6	15	11 391	11 466.6	42.0	14.8	10 917	11 332.4	29.2
D1	35	3	5	5	3 402	3 443.4	20.8	4.5	2 776	3 082.3	19.6
D2	46	4	5	6	4 231	4 263.0	22.6	6	4 101	4 128.4	20.8
D3	53	4	5	6.3	4 357	4 525.7	23.8	6.1	4 237	4 318.4	20.8
D4	61	3	5	8	5 728	5 778.4	28.7	8	5 495	5 648.7	22.8
D5	66	3	5	8.9	6 011	6 362.7	37.2	8.3	5 703	5 927.4	26.0
D6	70	4	5	9	6 288	6 330.1	31.5	9	6 196	6 256.9	24.9
D7	84	3	5	11	7 791	7 877.9	36.8	11	7 676	7 741.8	26.1
D8	108	5	14	13.9	9 882	10 276.6	47.5	13.5	9 669	10 014.0	29.5
D9	118	6	11	15	11 140	11 285.9	48.1	14.9	10 524	11 029.7	31.9
D10	121	6	16	15.7	11 583	11 944.3	51.5	15.6	11 406	11 778.9	33.8
D11	127	7	14	16	11 766	11 900.7	51.0	16	11 707	11 812.3	32.9
D12	131	7	15	17	12 653	12 821.3	51.4	16.7	11 988	12 502.9	33.7
D13	133	7	17	18	13 752	13 865.3	54.0	17.8	12 953	13 497.1	34.8
D14	135	4	14	17	12 840	12 894.3	53.8	16.9	12 286	12 718.4	35.2
D15	146	7	15	18.9	13 751	14 164.1	58.5	18.7	13 534	13 922.0	36.8
D16	157	6	14	18.5	13 330	13 643.5	57.7	18.4	13 211	13 478.2	40.2
D17	166	6	14	21	15 754	15 816.6	60.3	20.6	15 152	15 489.4	39.7
D18	173	7	15	21	15 547	15 650.5	67.7	21	15 489	15 608.0	44.3
D19	185	6	14	22	16 236	16 419.4	69.2	22	16 311	16 373.0	45.7
D20	185	6	15	21.7	15 726	16 118.1	65.7	21.9	15 623	16 137.5	42.6

Table 7

Example of a route in the solution on instance C20.

Request	45	90	45	90	45	90	11	10	10	11	10
Skip type	0	0	0	0	0	0	2	0	0	0	0
Location	1	1	10	10	7	4	4	4	9	9	1
Action	p	p	e	e	d	d	p	p	e	e	d
Load	1	2	2	2	1	0	1	2	2	2	1

Request	59	59	59	9	72	72	72	9	58	58	9	58
Skip type	4	0	4	4	0	0	0	0	0	0	4	0
Location	1	13	0	0	0	9	1	13	3	11	3	3
Action	p	e	d	p	p	e	d	e	p	e	d	d
Load	1	1	0	1	2	2	1	1	2	2	1	0

programs, and we have developed valid inequalities for both models. The two versions of the problem were solved heuristically by means of a VNS-inspired heuristic.

Extensive computational experiments were conducted on 80 real-life instances provided by our industrial partner, involving up to 185 requests. Our results indicate that solving the mathematical models by CPLEX only works for relatively small instances. In contrast, our heuristic can quickly yield high-quality solutions within modest computing times on larger instances. The quality of the solutions can be assessed by comparing them to exact solutions, when this is possible, in which case our heuristic always reaches an optimum. Otherwise, two measures of quality are the small coefficients of variations computed

Table 8

Results obtained by the ILP models for fixed and flexible return for the 17 instances with up to 20 requests.

	Fixed return				Flexible return			
	LB	UB	Time (s)	Heu	LB	UB	Time (s)	Heu
A1	661	661	2	661	648	648	10	648
A2	645	645	27	645	625	625	1329	625
A3	628	628	12	628	619	619	9	619
A4	720	720	1247	720	681.0	695	TL	695
A5	720.1	724	TL	724	684.2	716	TL	707
A6	1240	1240	172	1240	694.9	1216	TL	1202
A7	724.5	1248	TL	1248	668.8	1220	TL	1211
A8	1315.9	1356	TL	1356	1270.1	1323	TL	1314
A9	1249.4	1253	TL	1253	1217.0	1250	TL	1244
A10	1286.8	1316	TL	1316	1287.0	1298	TL	1297
A11	1279.0	1345	TL	1342	1235.0	1343	TL	1289
B1	1286.8	1333	TL	1323	1230.8	1296	TL	1299
B2	1123.9	1153	TL	1148	1082.0	1119	TL	1121
C1	864.4	1362	TL	1362	719.1	1304	TL	1297
C2	1481	1481	967	1481	1373.0	1461	TL	1438
C3	2254.7	2339	TL	2331	1646.5	2338	TL	2269
C4	1406.2	2051	TL	2034	1290.0	2003	TL	1520

over several runs, and comparisons with the best solution values found by CPLEX which almost always favor our heuristic.

Table A.9

Information about the root node relaxation for fixed return. Top: Root lower bound. Bottom: run time for solving the root node.

	All	Ex. (6)	Ex. (34)	Ex. (35)	Ex. (36)	Ex. (37)–(38)	Ex. (40)–(42)
A1	661	655.41	652.4	661	661	653.1	653.6
A2	619.3	619.0	618.1	619.0	619.1	618.1	595.9
A3	619	619	619	619	619	619	613
A4	683.7	682.7	678.5	684.6	683.7	680.1	651.5
A5	697.2	694	689	695	695.0	688.9	670.3
A6	1226	1224	1215	1225	1225.5	1224.8	1189.3
A7	674.3	674	671	674	674.3	676.3	652.7
A8	1303.6	1304	1296	1303	1303.4	1302.7	1264.8
A9	1242.9	1243	1238	1243	1243.1	1243.1	1208.8
A10	1283	1280	1274	1280	1280	1280	1214
A11	1278.8	1279	1253	1279	1278.8	1279	1175.5
B1	1276.8	1277	1272	1277	1276.4	1274.1	1239.2
B2	1119.8	1120	1116	1119	1119.8	1110.5	1081.4
C1	802.4	803	798	803	803.8	802.1	725.6
C2	1411.6	1411	1401	1414	1414.4	1388.8	1380.0
C3	2228.4	2228	2219	2227	2225.0	2216.5	2142.4
C4	1394.0	1390	1375	1386	1385.1	1386.7	1266.5
A1	0.6	0.9	0.5	0.6	0.6	0.4	0.9
A2	1.4	1.4	1.7	1.5	1.3	1.4	1.9
A3	1.3	1.1	1.5	1.2	1.3	1.3	2.2
A4	15.4	11.8	16.7	14.2	15.6	9.7	34.4
A5	11.7	9.5	12.7	12.5	11.3	11.1	16.9
A6	16.3	13.1	17.9	13.4	14.2	18.2	40.0
A7	12.8	13.7	16.9	14.2	12.8	12.7	37.4
A8	74.1	75.8	62.6	72.5	71.9	66.8	303.9
A9	123.5	116.7	119.5	90.2	69.4	100.1	314.1
A10	1263.9	440.5	1192.5	723.8	585.3	1131.6	3600.2
A11	775.7	461.3	699.6	673.2	735.9	562.1	1786.2
B1	158.6	135.9	206.8	174.0	144.9	118.2	173.0
B2	151.9	120.9	166.7	158.2	193.8	134.2	415.6
C1	8.4	8.4	8.0	8.6	8.7	7.4	16.4
C2	7.9	7.1	7.9	8.5	10.0	7.4	7.2
C3	96.1	65.2	74.9	87.0	81.8	79.4	280.8
C4	138.5	107.1	143.3	99.9	94.1	94.1	1545.2

Table A.10

Information about solution after 1 h run time. Top: Lower bound. Bottom: Run time (sec) or gap after 1 h in () if optimality is obtained.

	All	Ex. (6)	Ex. (34)	Ex. (35)	Ex. (36)	Ex. (37)–(38)	Ex. (40)–(42)
A1	661	661	661	661	661	661	661
A2	645	645	645	645	645	645	645
A3	628	628	628	628	628	628	628
A4	720	720	720	720	720	720	686.1
A5	724	716.5	713.7	718.4	724	724	704.6
A6	1240	1240	1240	1240	1240	1240	1230.2
A7	720.9	718.0	711.5	717.8	714.5	711	669.9
A8	1315.0	1319.5	1318.5	1314.2	1316.4	1316.3	1267.7
A9	1253	1253	1238.1	1253	1253	1248	1209.4
A10	1283.1	1284	1274.3	1283.6	1283.8	1283.3	1216.8
A11	1279.3	1278.8	1255.2	1278.8	1279.0	1279.1	1175.8
B1	1282.7	1281.1	1275.9	1281.3	1281.1	1278.2	1239.9
B2	1121.1	1121.0	1119.1	1121.3	1121.4	1113.3	1081.9
C1	850.9	853.1	841.0	855.0	854.9	851.6	773.8
C2	1481	1481	1481	1481	1481	1481	1464
C3	2250.7	2249.5	2243.8	2250.0	2250.4	2237.1	2148.6
C4	1412.9	1406.9	1399.9	1407.5	1404.2	1406.6	1268.9
A1	1.5	2	1.7	1.5	1.5	1.5	1.9
A2	23.3	26.5	58.1	19.9	17.9	54.4	237.5
A3	12.4	7.3	20	15.5	11.7	33.4	128
A4	835.3	603.6	2076.4	740.1	821.3	903.5	(4.9)
A5	1616.3	(1.1)	(1.4)	(0.8)	2821.6	2568.4	(2.8)
A6	275.1	138.1	1246.1	231.8	274.9	201	(0.8)
A7	(73.1)	(73.8)	(75.4)	(73.9)	(74.7)	(75.5)	(86.3)
A8	(3.1)	(2.8)	(2.8)	(3.2)	(3)	(3)	(7)
A9	2805.2	2530.3	(1.2)	2967.9	2951.4	(0.4)	(3.6)
A10	(2.6)	(2.5)	(3.5)	(2.5)	(2.5)	(2.6)	(8.2)
A11	(5.1)	(5.2)	(7.1)	(5.2)	(5.2)	(5.2)	(14.4)
B1	(3.9)	(4.1)	(4.7)	(4)	(4.1)	(4.3)	(7.5)
B2	(3.2)	(3.2)	(3.3)	(3.2)	(3.2)	(3.9)	(6.9)
C1	(60.1)	(59.7)	(62)	(59.3)	(59.3)	(59.9)	(76)
C2	376.1	449.2	911.3	421.8	533.8	1474.2	(1.2)
C3	(3.6)	(3.6)	(5.1)	(3.6)	(3.6)	(4.2)	(8.5)
C4	(44.2)	(44.9)	(45.3)	(44.8)	(45.1)	(44.9)	(60.6)

Table B.11

Information about the root node relaxation for flexible return. Top: Root lower bound. Bottom: run time for solving the root node.

	All	Ex. (6)	Ex. (34)	Ex. (35)	Ex. (36)	Ex. (37)+(39)	Ex. (40), (41),(43)
A1	619	619.4	619	619	619	619	597
A2	604	604	588	604	604	604	588.5
A3	618	618	616	618	618	618	606
A4	678	678	641	678	678	678	610
A5	656	656	639	656	656	656	612
A6	662	667	649	666	666	662	622
A7	655	655	634	655	655	655	608
A8	1266	1266	1238	1266	1266.1	1267.0	1216.5
A9	1217	1217	1191	1217	1217	1217	1156
A10	1281	1283	1227	1287	1280	1284	1176
A11	1234	1234	1176	1234	1227	1234	1155
B1	1227	1228	1219	1228	1228	1228	1200
B2	1082	1082	1073	1082	1081	1081	1041
C1	697.3	695.2	691.7	694.5	702.2	694.5	618.7
C2	1304.0	1306.5	1249	1305	1305	1306	1257
C3	1612	1625.5	1557.0	1612	1623	1622.9	1512
C4	1282	1282	1251	1282	1282	1282	1126.5
A1	0.3	0.4	0.3	0.4	0.4	0.4	0.3
A2	0.5	0.5	0.5	0.4	0.5	0.5	1.0
A3	0.4	0.4	0.5	0.4	0.4	0.4	0.4
A4	1.6	1.7	1.4	1.7	1.8	1.5	1.8
A5	2.4	2.2	2.3	2.4	2.3	2.2	2.3
A6	4.1	5.7	4.7	4.7	4.4	3.8	4.2
A7	4.1	3.8	4.0	4.5	4.6	4.0	4.6
A8	15.3	20.7	13.0	15.5	19.2	18.7	23.1
A9	14.9	13.5	11.7	15.0	15.2	15.2	12.2
A10	36.6	37.8	36.2	45.2	35.7	41.6	37.4
A11	46.0	47.9	41.9	44.7	42.9	44.7	50.8
B1	28.3	31.1	26.3	29.7	33.3	30.7	30.5
B2	29.5	27.0	26.6	29.9	29.4	27.0	28.6
C1	4.0	5.2	4.4	4.2	4.3	4.4	7.0
C2	5.4	5.0	4.5	5.6	5.3	5.7	7.3
C3	15.5	21.5	19.3	15.8	23.1	22.7	18.1
C4	24.9	24.7	37.4	26.9	26.7	25.9	41.1

Table B.12

Information about solution after 1 h run time. Top: Lower bound. Bottom: Run time (sec) or gap after 1 h in () if optimality is obtained.

	All	Ex. (6)	Ex. (34)	Ex. (35)	Ex. (36)	Ex. (37)+(39)	Ex. (40), (41),(43)
A1	648	648	648	648	648	648	648
A2	625	625	608.6	625	625	625	608.5
A3	619	619	619	619	619	619	619
A4	680	679.3	643.4	679	679.3	678	654.8
A5	676.5	678.5	659.7	679.7	673.6	671.3	622.3
A6	692.7	691	669.3	690.3	696	697	634
A7	659.1	660.9	641.3	662.3	666.3	660.8	626
A8	1270	1269	1240	1268.5	1273.3	1273.3	1225.9
A9	1217	1217	1191	1217	1217	1217	1171
A10	1284	1287	1227	1287	1287	1287	1191
A11	1234	1234	1176	1234	1234.3	1234	1155
B1	1230	1230	1223	1229.2	1230	1230.1	1209.4
B2	1083	1083	1073	1082.2	1082.3	1082.4	1050
C1	708.7	716.9	708.4	716.5	713.2	716.0	666.3
C2	1368.6	1376.2	1316.0	1372.1	1372.6	1365.7	1319.8
C3	1636.6	1639.8	1578	1649	1643.4	1642	1548.4
C4	1291.8	1294.4	1254.8	1291.5	1296.2	1290.1	1135.6
A1	10.4	19.5	19	31	12	16.1	29.6
A2	1322.2	347.4	(3.7)	906.4	542.8	1349	(2.7)
A3	9	8.1	17.5	8.2	8.9	7.4	231.5
A4	(2.2)	(2.3)	(8)	(2.4)	(2.3)	(2.5)	(6.8)
A5	(6)	(6.6)	(9.8)	(4.8)	(7.5)	(7.1)	(14.7)
A6	(73.4)	(73.8)	(79.5)	(74.1)	(72.7)	(72.3)	(91.5)
A7	(83.7)	(84)	(90.2)	(82.7)	(81.6)	(84.5)	(95.8)
A8	(5.4)	(6.9)	(6.3)	(3.7)	(3.4)	(3.3)	(8.7)
A9	(2.2)	(2.2)	(4.8)	(2.2)	(2.2)	(2.2)	(6.7)
A10	(1.1)	(1.9)	(6.8)	(1.8)	(1.9)	(1.1)	(10.7)
A11	(8.8)	(8.8)	(13.4)	(8.8)	(8.8)	(8.8)	(16.3)
B1	(7.8)	(7.4)	(8.4)	(7.5)	(7.4)	(7.8)	(8.8)
B2	(4)	(3.5)	(6.1)	(3.8)	(4.5)	(3.8)	(8.7)
C1	(84)	(83.6)	(85.8)	(82)	(84.5)	(83.8)	(97.5)
C2	(6.8)	(6.2)	(11)	(6.6)	(6.2)	(6.6)	(10.8)
C3	(41.5)	(41.8)	(47.4)	(41.1)	(42.1)	(41.7)	(50.2)
C4	(55.1)	(54.9)	(58.7)	(55.1)	(56.1)	(58.7)	(80.3)

Table C.13
Comparison between our heuristic and CPLEX for the fixed return case.

Fixed	CPLEX				Our heuristic	
	UB	LB	Gap	Time	UB	Time
A1	661	661	0	0.7	661	9
A2	645	645	0	18.4	645	11
A3	628	628	0	11.9	628	11.3
A4	720	720	0	2 505.0	720	13
A5	724	724	0	2 607.8	724	11.5
A6	1240	1240	0	517.1	1240	16.5
A7	1253	682.9	83.5	7 204.6	1248	16.7
A8	1356	1308.7	3.6	7 259.6	1356	18.1
A9	1262	1242.7	1.6	7 247.5	1253	18.9
A10	10 538	1280.0	723.3	7 330.5	1316	20.1
A11	4127	1278.7	222.8	7 329.6	1342	17.7
A12	No feasible	1387		7 569.4	1935	20.1
A13	No feasible	1385.0		7 573.7	1929	20.6
A14	No feasible	1834.0		8 209.1	1979	19.6
A15	No feasible	1987.3		8 297.6	2076	18.4
A16	No feasible	1981.3		10 075.7	2673	21.5
A17	No feasible	1997.9		10 616.0	2679	21.5
A18	No feasible	2079.8		11 205.9	2695	22.2
A19	No feasible	2597.5		12 602.1	2750	20.1
A20	No feasible	0		24 911.1	2745	24.1
B1	1362	1279.9	6.4	7 428.3	1323	20.2
B2	1190	1120.1	6.2	7 422.6	1148	21.3
B3	3385	1130.1	199.5	7 527.9	1188	22.4
B4	No feasible	1112.4		7 527.0	1178	22.4
B5	No feasible	1116.7		7 711.5	1171	23.8
B6	No feasible	1163.4		7 697.1	1205	21.5
B7	No feasible	1172.4		8 334.0	1761	24.1
B8	No feasible	1182.1		8 334.8	1759	24
B9	No feasible	1656.8		9 753.2	1741	25.8
B10	No feasible	1739.5		9 747.8	1846	24.2
B11	No feasible	1690.0		12 623.5	1812	22.7
B12	No feasible	1179.9		8 195.7	1934	24.1
B13	No feasible	0		19 130.9	2393	25.8
B14	No feasible	0		19 234.7	2390	25.4
C1	1362	807.3	68.7	7 200.8	1362	14.9
C2	1481	1481.0	0	881.5	1481	12.6
C3	2336	2235.8	4.5	7 230.3	2331	14.2
C4	5411	1397.7	287.1	7 273.3	2034	16.8
C5	No feasible	2220.4		7 473.4	2912	16.5
C6	No feasible	2875.7		7 469.4	3123	15.4
C7	No feasible	2798.4		8 174.7	3137	19
C8	No feasible	2948.6		9 231.7	3823	19.2
C9	No feasible	3706.7		10 549.8	4712	20.1
C10	No feasible	3586.9		12 503.7	4692	20.3
C11	No feasible	0		19 717.7	5451	24.6
C12	No feasible	0		19 604.5	5218	22.8
D1	No feasible	2642.4		9 708.4	3402	20.8
D2	No feasible	0		20 024.8	4231	22.6

We observe from our computational experiments that savings can be obtained by allowing flexible return of skips opposed to returning each skip to its origin. However, the flexibility also affects the daily operations for the recycling company negatively, as it requires a more centralized planning and a loss of autonomy for the drivers. Furthermore, the more complex patterns of actions under the flexible strategy increases the risk of errors for the drivers. Whether the savings are large enough to outweigh these matters must be left as a managerial decision.

CRedit authorship contribution statement

Sanne Wöhlk: Conceptualization, Methodology, Writing, Software.
Gilbert Laporte: Conceptualization, Methodology, Writing.

Acknowledgments

This project was funded by the Danish Council for Independent Research - Social Sciences, Denmark. Project ‘Transportation issues related to waste management’ [grant number 4182-00021] and by the Natural Sciences and Engineering Research Council of Canada [grant

number 2015-06189]. This support is gratefully acknowledged. Thanks are due to the referees for their valuable comments.

Appendix A. Exact solution of fixed return

In this appendix, we provide detailed results of the analysis of the strength of the valid inequalities in Section 3.3 for the fixed return problem. Using a time limit of one hour, we have run the model seven times with different configurations of the constraints. The first configuration consists of the base model (1)–(18) from Section 3.1 as well as all valid inequalities for this model: Constraints (34)–(38) and (40)–(42). In the second configuration, we exclude constraints (6), but keep all valid inequalities from Section 3.3, and in the remaining five configurations, we exclude one family of valid inequalities at a time, while keeping the remaining (and also keeping constraints (6)).

The results are shown in Tables A.9 and A.10, where Table A.9 shows results for the solution of the root node, and Table A.10 gives the results for one hour of computation time.

In Table A.9, the top part gives the lower bound of the root node and the lower part provides the computation time in seconds. The best bound for each instance is marked in bold. The most striking result in

Table C.14

Comparison between our heuristic and CPLEX for the flexible return case.

Flexible	CPLEX				Our heuristic	
	UB	LB	Gap	Time	UB	Time
A1	648	648	0	15.9	648	9.1
A2	625	597	4.7	7 200.2	625	10.9
A3	619	619	0	24.2	619	11.3
A4	695	610.3	13.9	7 200.8	695	12.9
A5	712	612	16.3	7 200.6	707	11
A6	1201	628	91.2	7 201.6	1202	16.3
A7	1216	608	100.0	7 205.1	1211	16.5
A8	1329	1207.3	10.1	7 207.2	1314	17.9
A9	1244	1159.6	7.3	7 210.9	1244	18.4
A10	1350	1176	14.8	7 271.7	1297	20.9
A11	No feasible	1155		7 380.2	1289	19.4
A12	No feasible	1189		7 459.3	1920	20.5
A13	No feasible	1195.1		7 456.7	1879	20.8
A14	No feasible	1150		7 678.9	1951	21.1
A15	No feasible	1792.5		7 725.4	2077	18.6
A16	No feasible	1774		8 306.6	2642	21.8
A17	3552	1771	100.6	8 412.5	2638	21.8
A18	No feasible	1888.2		8 655.3	2682	22.5
A19	No feasible	1815		9 145.1	2702	19.8
A20	5314	2280	133.1	12 842.9	3251	23.2
B1	1434	1203.8	19.1	7 353.7	1299	20.3
B2	1119	1044.5	7.1	7 336.8	1121	20.8
B3	No feasible	1039.1		7 382.9	1163	23
B4	No feasible	1030.5		7 373.4	1156	23
B5	No feasible	1052.8		7 440.3	1166	24.8
B6	No feasible	1050		7 450.2	1192	23.3
B7	No feasible	1056.1		7 682.1	1751	23.9
B8	No feasible	1067.6		7 660.7	1739	24
B9	No feasible	1068		8 166.4	1737	26.5
B10	No feasible	1142		8 174.4	1833	25.6
B11	4986	1574	216.8	9 015.7	1801	23.9
B12	No feasible	1070.8		7 606.6	1916	24.7
B13	No feasible	1609		10 798.4	2378	26.5
B14	No feasible	1592		10 535.4	2373	25.9
C1	1309	617.6	111.9	7 202.4	1297	14.2
C2	1447	1253.2	15.5	7 203.0	1438	13.2
C3	2342	1514.0	54.7	7 264.6	2269	15.4
C4	No feasible	1129.3		7 305.1	1520	15.9
C5	No feasible	1862.6		7 394.6	2810	16.2
C6	No feasible	1977.0		7 403.8	3035	15.3
C7	No feasible	1755.3		7 656.2	2899	14.2
C8	No feasible	2388.2		8 000.5	3568	17.5
C9	No feasible	2695.0		8 410.5	4498	17.7
C10	No feasible	2583.0		9 041.7	4487	17.7
C11	No feasible	3152.0		11 142.7	5312	19
C12	No feasible	3005.0		11 124.4	4458	17.2
D1	No feasible	1950.3		8 058.8	2776	19.6
D2	No feasible	2461.1		10 946.7	4101	20.8

this table is the large increase in computation time for the root node when the symmetry breaking constraints (40)–(42) are excluded from the model.

Table A.10 gives the results for one hour of computation time. Again, the top part provides the lower bound. The lower part gives the computation time in seconds for solving the problem to optimality. In the cases where this was not possible within the computation time limit, the gap after one hour is stated in parentheses. Again, the best results are marked in bold. From these results, we see that the three situations of excluding constraints (34), (37)–(38), or (40)–(42) do not lead to unique best performance after one hour of computation time.

From the two tables, we conclude that all the valid inequalities presented in Section 3.3 are indeed useful for some of the instances for the fixed return problem. However, not all inequalities are useful for all instances. For some instances (e.g. A10) the significantly longer solution time of the root node when including all inequalities is not outweighed by a better solution after one hour, whereas for other instances (e.g. C4) the better solution value of the root as well as after one hour make up for the longer root time.

Appendix B. Exact solution of flexible return

In this appendix, we provide detailed results of the analysis of the strength of the valid inequalities in Section 3.3 for the flexible return problem. We use the same approach as in Appendix A. For the flexible return problem, the first configuration consists of (19)–(33) from Section 3.2 and all valid inequalities for this model: Constraints (34)–(37), (39)–(41), and (43), and in the other configurations, one family of inequalities are excluded at a time. The results are presented in Tables B.11 and B.12, and follow the same logic as the tables in Appendix A.

It is harder to draw clear conclusions for the flexible return case than for the fixed return case. However, one conclusion that is clear from both Tables B.11 and B.12 is that constraints (34), (40), (41), and (43) should not be excluded from the model, as excluding them does not yield a unique best solution for any of the instances. Except for a few cases, constraints (6) can safely be excluded from the model. Among the other configurations, they each obtain the best root relaxation for nine to 14 instances, and the best result after one hour for six to nine instances. As there is no clear pattern in solution quality

among the configurations, we conclude that all the valid inequalities are indeed useful for some of the instances, but not necessarily for all.

Appendix C. Comparison between our heuristic and CPLEX

In this appendix, we compare our heuristic with the only existing heuristic for the problem, namely truncated CPLEX. In order to give CPLEX the best terms, we allow it to focus on finding feasible solutions by setting the parameter ‘MIPEmphasis’ to 1. Even though our heuristic runs in less than half a minute, we allow CPLEX to spend two hours of computation time. The comparison is shown in Tables C.13 and C.14 for fixed and flexible return, respectively. In each of the tables, the first four columns provide the results of running CPLEX for two hours and the last two columns repeat the best solution and run time for our heuristic from Tables 5 and 6 for ease of reading. For CPLEX, we provide the value of the best feasible solution obtained. If no feasible solution is found within the run time limit, this is stated. We also provide the lower bound, the optimality gap in the cases where both upper and lower bounds are available, and the computation time. Note that CPLEX sometimes exceeds its own run time limit due to the size of the model. Out of the 80 instances, we only show results for the 48 with at most 50 requests, because CPLEX was unable to obtain feasible solutions for larger instances. For the fixed return, there are six instances where CPLEX did not even succeed in solving the root node and thereby obtain a lower bound larger than zero. Both for the fixed and flexible return problem, CPLEX was only able for obtain a feasible solution within two hours of computation time for 18 instances.

For the fixed return case, CPLEX solved seven instances to optimality. For these instances, our heuristic obtained the same feasible solutions. For two additional instances CPLEX obtained the same feasible solutions as our heuristic. For all these instances, except one, CPLEX spent far more computation time. For the remaining nine instances where CPLEX obtained a feasible solution within two hours, our heuristic obtained a better solution in less than half a minute.

For the flexible return problem, CPLEX solved two instances to optimality, and our heuristic found the same solutions. For three additional instances CPLEX obtained the same feasible solutions as our heuristic. Again CPLEX spent more time to obtain these solutions. For two instances, CPLEX obtained slightly better solutions than our heuristic, and for the remaining 11 instances where CPLEX obtained a feasible solution within two hours, our heuristic obtained a better solution in less than half a minute.

References

- Andersson, H., Hoff, A., Christiansen, M., Hasle, G., Løkketangen, A., 2010. Industrial aspects and literature survey: Combined inventory management and routing. *Comput. Oper. Res.* 37, 1515–1536.
- Archetti, C., Bertazzi, L., 2021. Recent challenges in routing and inventory routing: E-commerce and last-mile delivery. *Networks* 77 (2), 255–268.
- Archetti, C., Mansini, R., Speranza, M.G., 2005. Complexity and reducibility of the skip delivery problem. *Transp. Sci.* 39, 182–187.
- Archetti, C., Speranza, M.G., 2004. Vehicle routing in the 1-skip collection problem. *J. Oper. Res. Soc.* 55, 717–727.
- Archetti, C., Speranza, M.G., 2005. Collection of waste with single load trucks: A real case. In: Fleischmann, B., Klose, A. (Eds.), *Distribution Logistics*. Springer-Verlag Berlin Heidelberg, pp. 105–119.
- Aringhieri, R., Bruglieri, M., Malucelli, F., Nonato, M., 2004. An asymmetric vehicle routing problem arising in the collection and disposal of special waste. *Electron. Notes Discrete Math.* 17, 41–47.
- Aringhieri, R., Bruglieri, M., Malucelli, F., Nonato, M., 2018. A special routing problem arising in the optimization of waste disposal: A real case. *Transp. Sci.* 52, 277–299.
- Baldacci, R., Bodin, L.D., Mingozzi, A., 2006. The multiple disposal facilities and multiple inventory locations rollon-rolloff vehicle routing problem. *Comput. Oper. Res.* 33, 2667–2702.
- Battarra, M., Cordeau, J.-F., Iori, M., 2014. Pickup-and-delivery problems for goods transportation. In: Toth, P., Vigo, D. (Eds.), *Vehicle Routing: Problems, Methods, and Applications*, second ed. In: MOS-SIAM Series on Optimization, pp. 166–191.
- Benjamin, A.M., Beasley, J.E., 2010. Metaheuristics for the waste collection vehicle routing problem with time windows, driver rest period and multiple disposal facilities. *Comput. Oper. Res.* 37, 2270–2280.
- Berbeglia, G., Cordeau, J.-F., Gribkovskaia, I., Laporte, G., 2007. Static pickup and delivery problems: A classification scheme and survey. *TOP* 15, 1–31.
- Bodin, L.D., Mingozzi, A., Baldacci, R., Ball, M.O., 2000. The rollon-rolloff vehicle routing problem. *Transp. Sci.* 34, 271–288.
- Bogh, M.B., Mikkelsen, H., Wöhlk, S., 2014. Collection of recyclables from cubes — A case study. *Socio-Econ. Plan. Sci.* 48, 127–134.
- Clarke, G., Wright, J.W., 1964. Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.* 12, 568–581.
- Coelho, L.C., Cordeau, J.-F., Laporte, G., 2014. Thirty years of inventory routing. *Transp. Sci.* 48, 1–19.
- Corberán, Á., Eglese, R., Hasle, G., Plana, I., Sanchis, J.M., 2021. Arc routing problems: A review of the past, present, and future. *Networks* 77 (1), 88–115.
- Corberán, Á., Laporte, G., 2015. *Arc Routing: Problems, Methods, and Applications*. In: SIAM Monographs on Discrete Mathematics and Applications, Philadelphia.
- Cordeau, J.-F., 2006. A branch-and-cut algorithm for the dial-a-ride problem. *Oper. Res.* 54, 573–586.
- De Maio, A., Laganá, D., Musmanno, R., Vocaturo, F., 2021. Arc routing under uncertainty: Introduction and literature review. *Comput. Oper. Res.* 135, 105442.
- De Meulemeester, L., Laporte, G., Louveaux, F.V., Semet, F.J., 1997. Optimal sequencing of skip collections and deliveries. *J. Oper. Res. Soc.* 48, 57–64.
- Derigs, U., Pullman, M., Vogel, U., 2013. A short note on applying a simple LS/LNS-based metaheuristic to the rollon-rolloff vehicle routing problem. *Comput. Oper. Res.* 40, 867–872.
- Ekeröth, T., 2019. *Genbrugspladserne Er Blandt Danmarks Største Attraktioner*. Dansk kommuner.dk.
- Elbek, M., 2018. *Modeling and Optimization of Collection Problems* (Ph.D. thesis). Aarhus University, Denmark.
- Elbek, M., Wöhlk, S., 2016. A variable neighborhood search for the multi-period collection of recyclable materials. *European J. Oper. Res.* 249, 540–550.
- Han, H., Ponce-Cueto, E., 2015. Waste collection vehicle routing problem: Literature review. *Traffic Eng. Rev.* 27, 345–358.
- Hauge, K., Larsen, J., Lusby, R.M., Krapp, E., 2014. A hybrid column generation approach for an industrial waste collection routing problem. *Comput. Ind. Eng.* 71, 10–20.
- Kiilerich, L., Wöhlk, S., 2018. New large-scale data instances for CARP and variations of CARP. *INFOR: Inf. Syst. Oper. Res.* 56, 1–32.
- Li, H., Jian, X., Chang, X., Lu, Y., 2018. The generalized rollon-rolloff vehicle routing problem and savings-based algorithm. *Transp. Res. B* 113, 1–23.
- Li, H., Lv, T., Lu, Y., 2016. The combination truck routing problem: A survey. *Procedia Eng.* 137, 639–648.
- Lu, J.W., Chang, N.B., Liao, L., Liao, M.Y., 2017. Smart and green urban solid waste collection systems: Advances, challenges, and perspectives. *IEEE Syst. J.* 11, 2804–2817.
- Mourão, M.C., Pinto, L.S., 2017. An updated annotated bibliography on arc routing problems. *Networks* 70, 144–194.
- Parragh, S.N., Doerner, K.F., Hartl, R.F., 2008. A survey on pickup and delivery problems, part II: Transportation between pickup and delivery locations. *J. Betriebswirtschaft* 58, 81–117.
- Rabbani, M., Tabrizi, F., Haeri, F., Farrokhi-Asi, H., 2016. A hybrid algorithm for solving a roll-on roll-off waste collection vehicle routing problem considering waste separation and recycling center. *Int. J. Appl. Oper. Res.* 6 (2), 19–31.
- Raucq, J., Sörensen, K., Cattrysse, D., 2019. Solving a real-life roll-on-roll-off waste collection problem with column generation. *J. Veh. Routing Algorithms* 2, 41–54.
- Richard, N., Richard, A.-A., Yeboah, A.B., Ahwiren, A.K., 2021. Household solid waste management in La-Nkwantanang-Midina municipality of the Greater Accra region, Ghana. *J. Environ. Stud.* 24, 1–14.
- Tonny, O., Emmanuel, S., 2020. Adoption of critical success factors for effective waste management in Ghana: A case of Ga-West municipality. *Int. J. Sci. Eng. Res.* 11 (4), 1836–1845.
- Wy, J., Kim, B.I., 2013. A hybrid metaheuristic approach for the rollon-rolloff vehicle routing problem. *Comput. Oper. Res.* 40, 1947–1952.
- Wy, J., Kim, B.I., Kim, S., 2013. The rollon-rolloff waste collection problem with time windows. *European J. Oper. Res.* 224, 466–476.
- Yusuf, A.A., Peter, O., Hassan, A.S., Tunji, L.A., Oyagbola, I.A., Mustafa, M.M., Yusuf, D.A., 2019. Municipality solid waste management system for Mukono district, Uganda. *Procedia Manuf.* 35, 613–622.
- Zbib, H., Wöhlk, S., 2019. A comparison of the transport requirements of different curbside waste collection systems in Denmark. *Waste Manage.* 87, 21–32.